

**ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN**



**ĐỒ ÁN GIỮA KỲ
HỌC PHẦN: HỆ ĐIỀU HÀNH MÃ NGUỒN MỞ**

TÊN CHỦ ĐỀ:

TÌM HIỂU VỀ LINUX KERNEL

Tên nhóm lớp: Nhóm 06 – Tên lớp: DCT124C6

Sinh viên thực hiện: Phòng Gia Hào – 3124411080

Giảng viên hướng dẫn: TS. Trịnh Tấn Đạt

Thành phố Hồ Chí Minh, tháng 6 năm 2026

LỜI CAM ĐOAN

Em là Phòng Gia Hào và nhóm của mình xin chân thành cảm ơn đến thầy Trịnh Tấn Đạt, giảng viên học phần Hệ điều hành mã nguồn mở đã giúp chúng em có được những kiến thức bổ ích để có thể ứng dụng vào thực tiễn một cách dễ dàng hơn.

Em cùng với nhóm rất mong thầy và các bạn sẽ góp ý phần đồ án của nhóm chúng em để chúng em có thể tiếp tục làm tốt hơn nữa. Nếu trong quá trình nhóm em làm đồ án mà có sai sót, chúng em xin chịu trách nhiệm về những gì mình đã làm.

MỤC LỤC

LỜI CAM ĐOAN.....	i
MỤC LỤC.....	ii
DANH MỤC CÁC HÌNH ẢNH.....	v
LỜI MỞ ĐẦU.....	1
CHƯƠNG 1. GIỚI THIỆU VỀ HỆ ĐIỀU HÀNH LINUX.....	3
1.1. Lịch sử ra đời và phát triển của Linux.....	3
1.2. Giới thiệu một số bản phân phối phổ biến của Linux	4
1.2.1. Linux Ubuntu	4
1.2.2. Linux Mint	6
1.2.3. Linux CentOS	7
1.2.4. Linux Debian	8
1.3. Giới thiệu cấu hình và cách cài đặt Linux	10
1.3.1. Cấu hình của Linux.....	10
1.3.2. Cách cài đặt Linux	10
1.4. Ưu điểm và khuyết điểm của Linux.....	11
1.4.1. Ưu điểm của Linux	11
1.4.2. Khuyết điểm của Linux.....	12
CHƯƠNG 2. TỔNG QUAN KIẾN TRÚC CỦA LINUX KERNEL.....	13
2.1. Khái niệm, chức năng của Linux Kernel.....	13
2.2. Lịch sử phát triển của Linux Kernel.....	15
2.3. Kiến trúc của Linux Kernel.....	17
2.3.1. Đặc điểm.....	17

2.3.2. Thành phần chính trong Linux Kernel.....	17
2.3.3. Linux Kernel thuộc kiến trúc Monolithic.....	19
2.3.3.1. Tìm hiểu các loại kiến trúc Kernel.....	19
2.3.3.2. Lý do Linux Kernel thuộc kiến trúc Monolithic.....	22
2.4. Cơ chế hoạt động trong Linux Kernel.....	23
2.4.1. Quá trình khởi động.....	23
2.4.2. Cách kernel giao tiếp với phần cứng.....	24
2.4.3. Cơ chế System Call.....	24
2.4.4. Context Switching.....	25
2.5. Ưu điểm và khuyết điểm của Linux Kernel.....	28
2.5.1. Ưu điểm của Linux Kernel.....	28
2.5.2. Khuyết điểm của Linux Kernel.....	28
CHƯƠNG 3. BẢO MẬT TRONG LINUX KERNEL.....	29
3.1. Khái niệm và chức năng về bảo mật trong Linux Kernel.....	29
3.1.1. Khái niệm về bảo mật trong Linux Kernel.....	29
3.1.2. Chức năng về bảo mật trong Linux Kernel.....	29
3.2. Một số nguyên tắc bảo mật Linux Kernel.....	30
3.2.1. Đặc quyền tối thiểu.....	30
3.2.2. Cập nhật và bản vá.....	30
3.2.3. Cách ly tiến trình.....	31
3.2.4. Phòng thủ nhiều lớp.....	32
3.3. Một số cách bảo mật trong Linux Kernel.....	33
3.3.1. Chặn truy cập bằng Firewall.....	33
3.3.2. Chú ý cập nhật thường xuyên.....	34

3.3.3. Sử dụng SFTP thay vì FTP	34
CHƯƠNG 4. ỨNG DỤNG THỰC TẾ CỦA LINUX KERNEL.....	35
4.1. Xu hướng phát triển của Linux Kernel.....	35
4.2. Linux Kernel trong Server.....	36
4.3. Linux Kernel trong Embedded System.....	38
4.4. Linux Kernel trong IoT.....	41
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	44
5.1. Kết luận.....	44
5.2. Hạn chế.....	44
5.3. Hướng phát triển.....	45
TÀI LIỆU THAM KHẢO	46

DANH MỤC CÁC HÌNH ẢNH

Hình 1.1. Chim cách cụt Tux – biểu tượng của hệ điều hành Linux.....	3
Hình 1.2. Chân dung của Linus Torvalds, người sáng lập Linux.....	4
Hình 1.3. Biểu tượng của Ubuntu.....	5
Hình 1.4. Giao diện màn hình của Ubuntu.....	5
Hình 1.5. Chân dung ông Mark ShuttleWorth - người lập ra Ubuntu.....	6
Hình 1.6. Biểu tượng của Linux Mint.....	6
Hình 1.7. Giao diện màn hình của Linux Mint.....	7
Hình 1.8. Biểu tượng của CentOS.....	7
Hình 1.9. Giao diện màn hình của CentOS.....	8
Hình 1.10. Biểu tượng của Debian.....	9
Hình 1.11. Giao diện màn hình của Debian.....	9
Hình 1.12. Cấu hình của hệ điều hành Linux.....	10
Hình 2.1. Quản lý tiến trình của Linux Kernel.....	14
Hình 2.2. Quản lý hệ thống tập tin trong nhân Linux.....	15
Hình 2.3. Linux 0.01 trong QEMU.....	16
Hình 2.4. Linux 0.11.....	17
Hình 2.5. Kiến trúc của Linux Kernel.....	18
Hình 2.6. Kiến trúc Microkernel.....	21
Hình 2.7. Kiến trúc Monolithic.....	21
Hình 2.8. Kiến trúc Hybrid.....	22
Hình 2.9. Sơ đồ các bước khởi động kernel.....	23
Hình 2.10. Cơ chế System Call.....	25
Hình 2.11. Process Context Switch.....	26

Hình 2.12. Thread Context Switch.....	27
Hình 2.13. Mode Switch.....	27
Hình 3.1. Chức năng phân quyền và kiểm soát truy cập trong bảo mật Linux Kernel.....	29
Hình 3.2. Nguyên tắc đặc quyền tối thiểu	30
Hình 3.3. Cách ly tiến trình.....	31
Hình 3.4. Phòng thủ nhiều lớp trong Linux Kernel.....	32
Hình 3.5. Bảo mật Linux bằng cách chặn truy cập bằng Firewall	33
Hình 3.6. Cách dùng SFTP thay FTP trong bảo mật Linux Kernel.....	34
Hình 4.1. Ngôn ngữ Rust	36
Hình 4.2. Câu lệnh để kiểm tra phiên bản Kernel trong Server.....	37
Hình 4.3. Câu lệnh hiển thị thông tin phiên bản Linux Kernel.....	37
Hình 4.4. Hệ thống nhúng (Embedded System).....	41
Hình 4.5. Máy đo điện tâm đồ (ECG).....	41
Hình 4.6. Internet vạn vật (Internet of Things - IoT).....	43

LỜI MỞ ĐẦU

Trong bối cảnh công nghệ thông tin phát triển mạnh mẽ, hệ điều hành đóng vai trò quan trọng trong việc quản lý và điều phối hoạt động của các thiết bị máy tính. Trong số các hệ điều hành hiện nay, Linux là một hệ điều hành mã nguồn mở phổ biến, được sử dụng rộng rãi trên máy chủ, máy tính cá nhân, thiết bị di động và các hệ thống nhúng. Trái tim của hệ điều hành Linux chính là Linux Kernel – thành phần cốt lõi chịu trách nhiệm quản lý tài nguyên phần cứng và cung cấp các dịch vụ thiết yếu cho các chương trình ứng dụng.

Linux Kernel được đánh giá cao nhờ tính ổn định, hiệu năng mạnh mẽ, khả năng bảo mật tốt và tính linh hoạt cao. Chính vì vậy, việc tìm hiểu về Linux Kernel không chỉ giúp nâng cao kiến thức về hệ điều hành mà còn tạo nền tảng quan trọng cho việc nghiên cứu, phát triển phần mềm, quản trị hệ thống và bảo mật thông tin.

Đề tài “Tìm hiểu về Linux Kernel” được thực hiện nhằm nghiên cứu cấu trúc, cơ chế hoạt động và các chức năng chính của nhân Linux. Qua đó, giúp người đọc hiểu rõ hơn về cách thức mà Linux quản lý tiến trình, bộ nhớ, thiết bị và các cơ chế bảo mật cũng như những ứng dụng thực tế mà Linux Kernel mang lại.

Đề án được trình bày trong 5 chương:

- Chương 1. Giới thiệu về hệ điều hành Linux. Chương 1 giới thiệu về những điều cơ bản nhất về hệ điều hành Linux, đó là sơ lược lịch sử ra đời và phát triển của Linux, một số bản phân phối của Linux, cấu hình và cách cài đặt Linux cũng như ưu điểm và khuyết điểm của hệ điều hành Linux.
- Chương 2. Tổng quan kiến trúc của Linux Kernel. Chương 2 giới thiệu về những điều cơ bản nhất về kiến trúc của Linux Kernel, đó là khái niệm và chức năng của Linux Kernel, lịch sử phát triển của Linux Kernel, giới thiệu về kiến trúc của Linux Kernel, cơ chế hoạt động của Linux Kernel và ưu điểm, khuyết điểm của Linux Kernel.
- Chương 3. Bảo mật trong Linux Kernel. Chương 3 giới thiệu về khái niệm

và chức năng bảo mật trong Linux Kernel, một số nguyên tắc bảo mật Linux Kernel và một số cách bảo mật trong Linux Kernel.

- Chương 4. Ứng dụng thực tế của Linux Kernel. Chương 4 giới thiệu về xu hướng phát triển của Linux Kernel trong thực tế và cũng như những ứng dụng thực tế trong server, embedded system và IoT.
- Chương 5. Kết luận và hướng phát triển. Chương 5 tập trung vào việc kết luận, tổng kết đồ án đã làm được những gì, hạn chế và hướng phát triển.

CHƯƠNG 1. GIỚI THIỆU VỀ HỆ ĐIỀU HÀNH LINUX

1.1. LỊCH SỬ RA ĐỜI VÀ PHÁT TRIỂN CỦA LINUX

- Linux là một họ các hệ điều hành tự do nguồn mở tương tự Unix dựa trên nhân Linux, một hạt nhân hệ điều hành được phát hành lần đầu tiên vào ngày 17 tháng 9 năm 1991 bởi Linus Torvalds. Mặc dù có khá nhiều tranh cãi về việc phát âm Linux, nhưng theo Linus chia sẻ: “Tôi không quá bận tâm việc mọi người phát âm tên tôi như thế nào, nhưng Linux luôn là Lih-nix”. Linux thường được đóng gói thành các bản phân phối Linux.
- Năm 1991, khi theo học tại đại học Helsinki (Phần Lan), Torvalds trở nên tò mò về hệ điều hành. Thất vọng vì việc cấp phép MINIX, lúc đó chỉ giới hạn sử dụng cho mục đích giáo dục, ông bắt đầu làm việc với nhân hệ điều hành của chính mình, cuối cùng trở thành Linux.
- Linus Torvalds đã muốn đặt tên cho sang chế của mình là “Freax”, một cách chơi chữ khi ghép các từ “free”, “freak” và “x”. Torvalds đã từng xem xét cái tên “Linux”, nhưng ban đầu bác bỏ nó vì cho rằng như thế là quá tự cao tự đại.
- Ari Lemmke, bạn học của Torvalds tại Helsinki University of Technology, một trong những quản trị viên tình nguyện của máy chủ FTP tại thời điểm đó, không nghĩ rằng “Freax” là một cái tên hay. Vì vậy, ông đã đặt tên dự án là “Linux” trên máy chủ mà không hỏi ý kiến của Torvalds nhưng sau đó Torvalds chấp thuận với tên Linux.
- Từ một hệ điều hành đơn giản để tìm hiểu về bộ vi xử lý Intel 386, Linux đã phát triển thành hạt nhân thống trị thế giới phần mềm, chạy trên mọi thứ từ các thiết bị thông minh đến các siêu máy tính mạnh nhất.



Hình 1.1. Chim cánh cụt Tux - biểu tượng của hệ điều hành Linux



Hình 1.2. Chân dung của Linus Torvalds, người sáng lập Linux

1.2. GIỚI THIỆU MỘT SỐ BẢN PHÂN PHỐI PHỔ BIẾN CỦA LINUX

Trong hệ điều hành Linux, có rất nhiều phiên bản nhưng phải kể đến là Ubuntu, Linux Mint, Puppy Linux,... Trong phần này, chúng ta chỉ giới thiệu 4 phiên bản của Linux là Ubuntu, Linux Mint, CentOS và Debian.

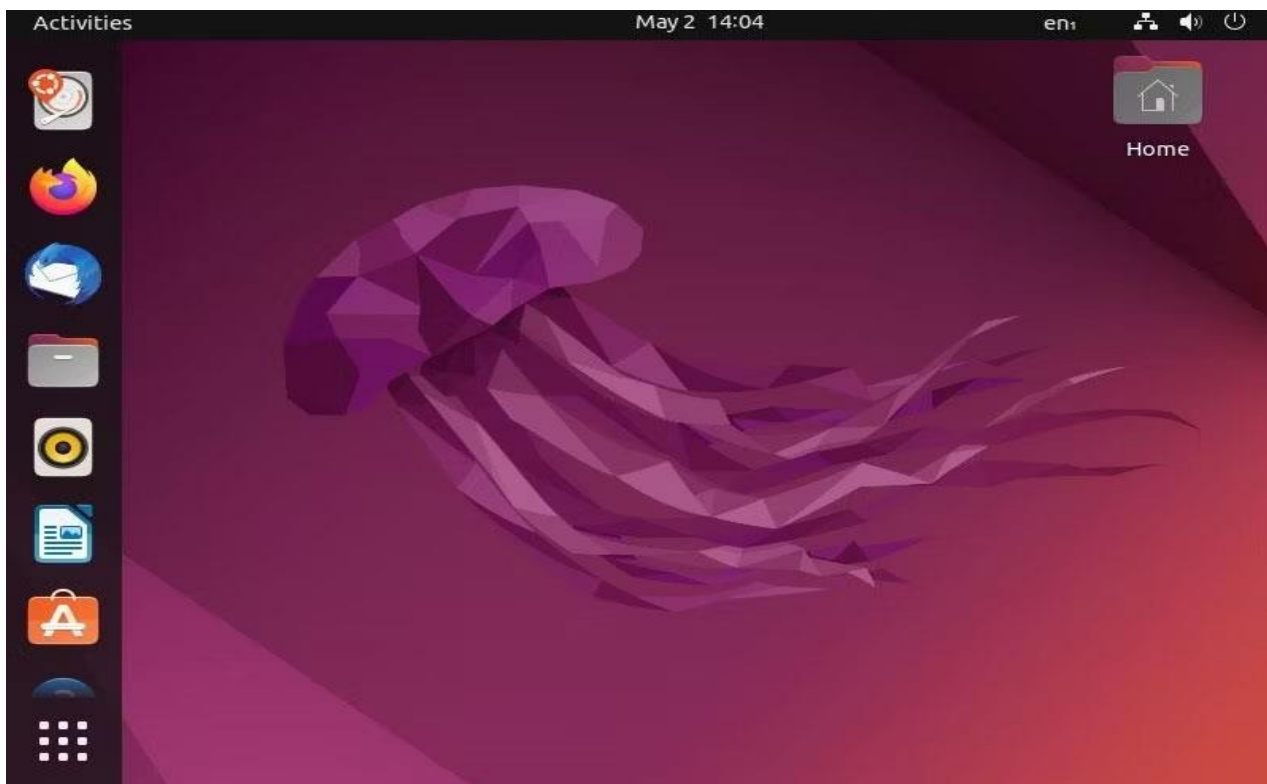
1.2.1. LINUX UBUNTU

- Ubuntu là một hệ điều hành mã nguồn mở dựa trên Linux, được phát triển bởi công ty Canonical.
- Ubuntu được ra đời vào năm 2004, sáng lập bởi Mark ShuttleWorth, một doanh nhân người Nam Phi. Ubuntu được xây dựng dựa trên Debian – một bản Linux khác nhưng được tối ưu hóa hơn về mặt giao diện và trải nghiệm người dùng.
- Kể từ khi ra mắt, Ubuntu đã phát hành một phiên bản mới mỗi sáu tháng. Mỗi phiên bản đều mang đến các tính năng mới, cải thiện hiệu suất và bảo mật.
- Ubuntu thường dành cho các đối tượng như lập trình viên, chuyên gia hệ thống, nhà nghiên cứu hoặc thậm chí là người dùng phổ thông thích vọc vạch.
- Một số cột mốc quan trọng:
 - + Ubuntu 4.10 “Warty Warthog” (2004): Phiên bản đầu tiên của Ubuntu.
 - + Ubuntu 10.04 “Lucid Lynx” (2010).

- Các tính năng nổi bật của Ubuntu:
- + Thừa hưởng tính năng đặc biệt của Linux.
- + Hỗ trợ trong quá trình cài đặt.
- + Giao diện thân thiện.
- + Tốn rất ít tài nguyên phần cứng.



Hình 1.3. Biểu tượng của Ubuntu



Hình 1.4. Giao diện màn hình của Ubuntu



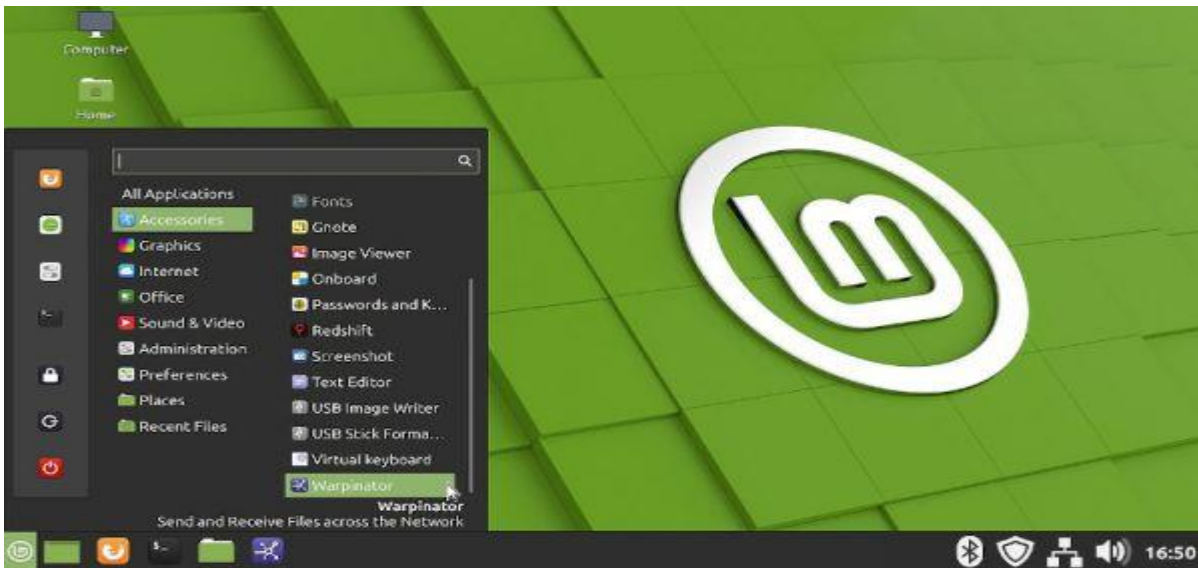
Hình 1.5. Chân dung ông Mark ShuttleWorth - người lập ra Ubuntu

1.2.2. LINUX MINT

- Linux Mint là một bản phân phối của Linux dựa trên nền tảng Ubuntu. Linux Mint có thêm nhiều tính năng mà Ubuntu không có như nhiều phần mềm cài đặt sẵn.
- Linux Mint có nhiều phiên bản dựa trên Ubuntu và Debian, sử dụng nhiều môi trường làm việc khác nhau theo từng phiên bản.
- Linux Mint rất lý tưởng cho người mới làm quen với Linux, người dùng muốn thay thế Windows hoặc MacOS miễn phí và chủ sở hữu máy tính cũ.



Hình 1.6. Biểu tượng của Linux Mint



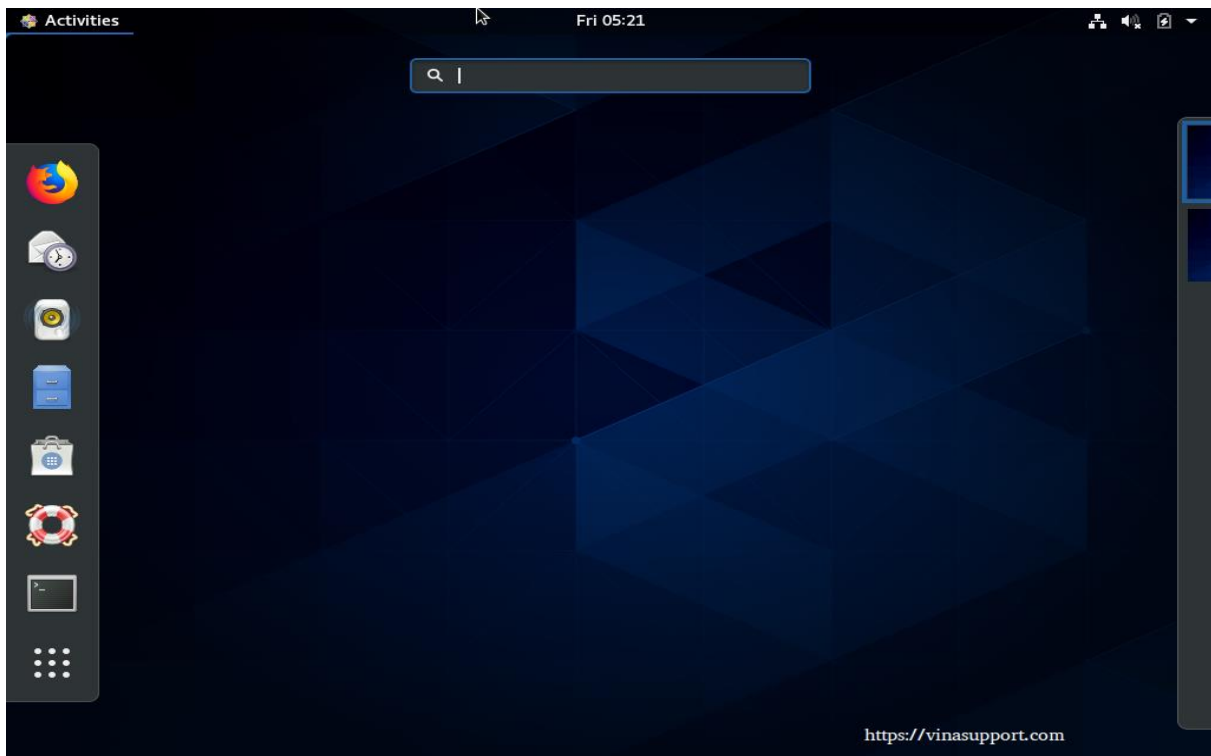
Hình 1.7. Giao diện màn hình của Linux Mint

1.2.3. LINUX CENTOS

- CentOS là một bản phân phối của Linux. Nó có nguồn gốc hoàn toàn từ bản phân phối Red Hat Enterprise Linux.
- Trước khi được biết đến dưới tên hiện tại, CentOS có nguồn gốc là một sản phẩm của CAOS Linux, được khởi động bởi Gregory Kurtzer.
- CentOS thường được dùng làm nền tảng máy chủ (server) và các đối tượng sử dụng chủ yếu bao gồm quản trị viên hệ thống, doanh nghiệp, tổ chức, nhà phát triển, nhà cung cấp dịch vụ Hosting/VPS và kỹ sư DevOps.



Hình 1.8. Biểu tượng của CentOS



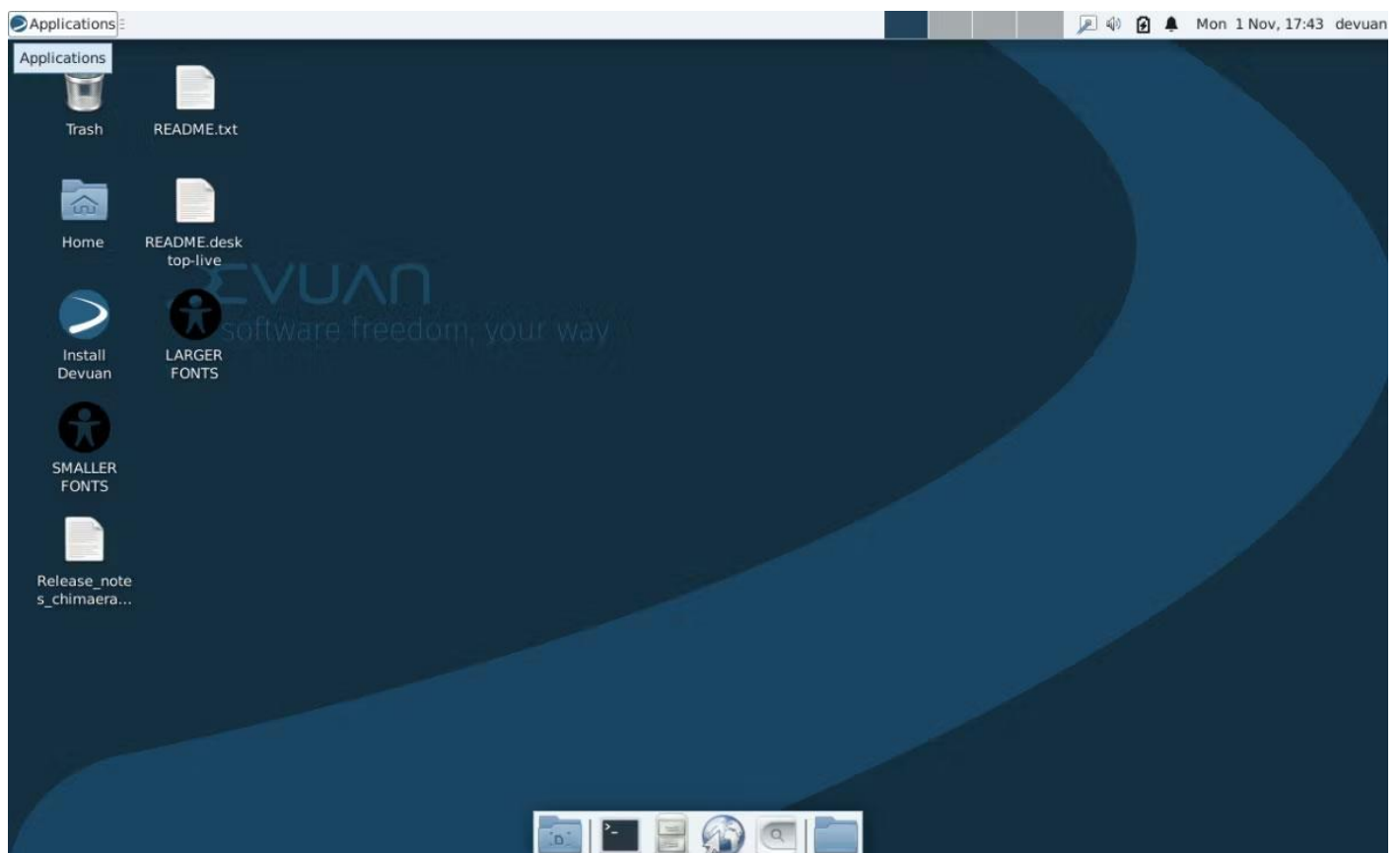
Hình 1.9. Giao diện màn hình của CentOS

1.2.4. LINUX DEBIAN

- Debian là một bản phân phối của hệ điều hành Linux được cấu thành hoàn toàn từ phần mềm tự do, được phát triển từ sự cộng tác của các tình nguyện viên trên khắp thế giới trong dự án Debian.
- Debian nổi tiếng với hệ thống quản lý gói của nó, mà cụ thể APT (Advanced Packaging Tool – Công cụ quản lý gói cao cấp), chính sách nghiêm ngặt đối với chất lượng các gói và bản phát hành, cũng như tiến trình phát triển và kiểm tra mở. Cách thức làm việc này đã giúp cho việc nâng cấp giữa các bản phát hành và việc cài đặt hay gỡ bỏ các gói phần mềm được dễ dàng hơn.
- Nhờ tính linh hoạt và bảo mật cao, các đối tượng sử dụng Debian thường là các quản trị viên hệ thống, lập trình viên, doanh nghiệp và người dùng Linux có kinh nghiệm.



Hình 1.10. Biểu tượng của Debian



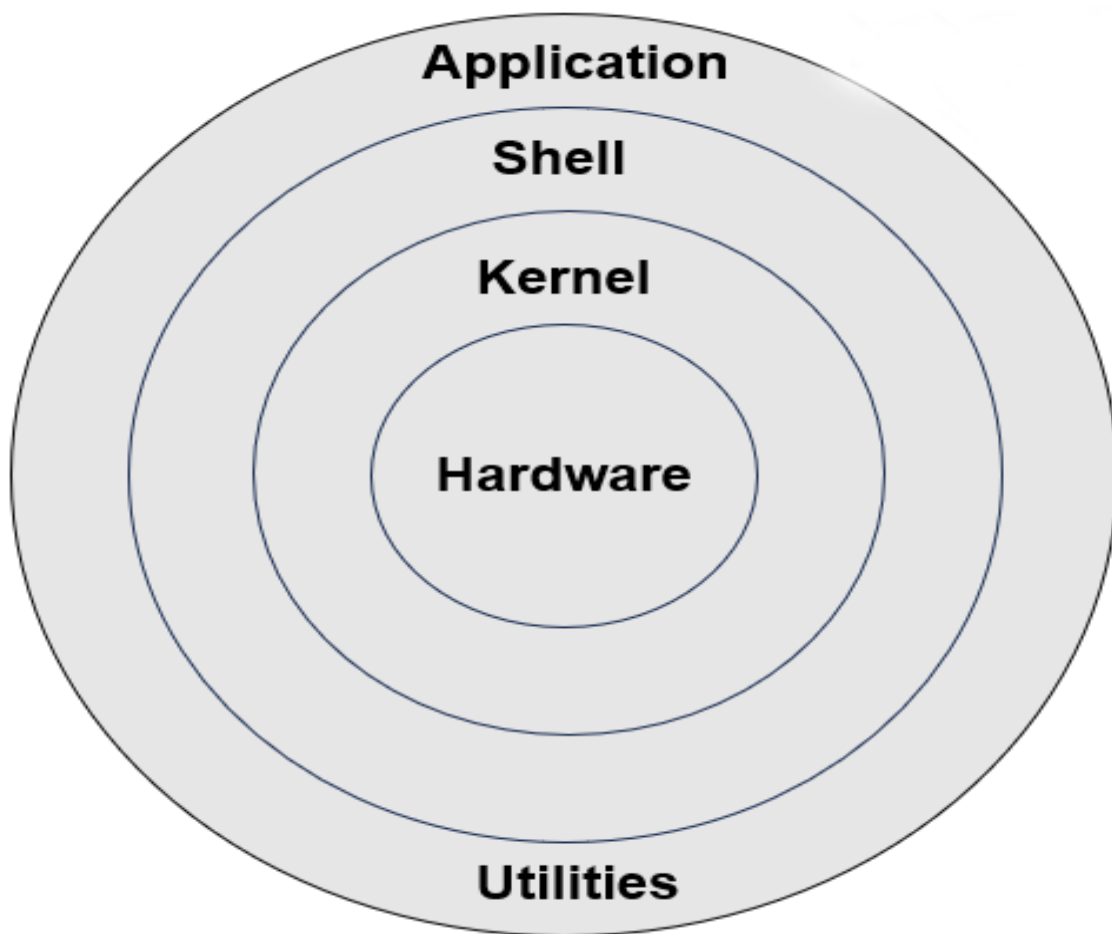
Hình 1.11. Giao diện màn hình của Debian

1.3. GIỚI THIỆU CẤU HÌNH VÀ CÁCH CÀI ĐẶT LINUX

1.3.1. CẤU HÌNH CỦA LINUX

Đối với Linux, cấu hình sẽ có các thành phần cốt lõi:

- Nhân (Kernel).
- Giao diện dòng lệnh (Shell).
- Phần cứng.
- Các ứng dụng và thư viện hệ thống.



Hình 1.12. Cấu hình của hệ điều hành Linux

1.3.2. CÁCH CÀI ĐẶT LINUX

- Một số lưu ý trước khi cài đặt Linux:
- + Kiểm tra cấu hình hệ thống.
 - CPU: Ít nhất là 1 GHz với 64-bit.

- RAM: tối thiểu 2GB, nhưng 4GB hoặc cao hơn sẽ giúp hệ thống chạy mượt mà hơn.
 - Dung lượng ổ cứng: Tối thiểu 25GB cho hệ điều hành và các phần mềm cơ bản.
 - Card đồ họa: Hỗ trợ đồ họa 3D sẽ giúp bạn có trải nghiệm sử dụng giao diện người dùng tốt hơn.
- + Tạo USB Bootable.
- + Sao lưu dữ liệu.
- Các bước thực hiện cài đặt:
- + Bước 1: Khởi động từ USB.
- + Bước 2: Quá trình cài đặt:
- Chọn ngôn ngữ và khu vực.
 - Cài đặt kết nối mạng.
 - Chọn kiểu cài đặt: Cài đặt song song hoặc cài đặt toàn bộ ổ cứng.
 - Phân vùng ổ cứng.
 - Tạo tài khoản người dùng.
 - Bắt đầu cài đặt.
- + Bước 3: Khởi động lại và hoàn tất cài đặt.
- Sau khi cài đặt xong, chúng ta có thể thực hiện các bước tiếp theo để thiết lập và tối ưu hóa hệ thống:
- + Cập nhật hệ thống.
- + Cài đặt các phần mềm cần thiết.
- + Cài đặt driver.

1.4. ƯU ĐIỂM VÀ KHUYẾT ĐIỂM CỦA LINUX

1.4.1. ƯU ĐIỂM CỦA LINUX

- Mã nguồn mở và miễn phí: Linux là một hệ điều hành mã nguồn mở, cho phép người dùng xem, sửa đổi và phân phối mã nguồn. Điều này mang lại sự linh hoạt và tự do cho người dùng, đồng thời loại bỏ chi phí bản quyền.
- Khả năng tùy biến: Linux cho phép người dùng tùy chỉnh hệ điều hành theo ý muốn của mình, từ giao diện người dùng đến các tính năng và ứng dụng. Điều này giúp người dùng có

thể tạo ra một môi trường làm việc phù hợp nhất với nhu cầu của họ.

- Tiết kiệm tài nguyên: Linux thường sử dụng ít tài nguyên hệ thống hơn so với các hệ điều hành khác, điều này có nghĩa là nó có thể hoạt động hiệu quả trên cả các máy tính có cấu hình thấp.
- Hiệu suất tốt cho server và hệ thống nhúng: Nhờ tính ổn định và khả năng xử lý tác vụ nặng, Linux thường được triển khai trên máy chủ web, cơ sở dữ liệu và các thiết bị hệ thống nhúng. Hiệu suất cao và mức tiêu thụ tài nguyên thấp giúp Linux duy trì vị thế hàng đầu trong lĩnh vực này.
- Hỗ trợ mạnh mẽ cho lập trình và phát triển phần mềm: Linux tích hợp nhiều công cụ và môi trường phát triển phần mềm hàng đầu, từ các trình biên dịch như GCC đến hệ thống quản lý container như Docker. Đây là hệ điều hành lý tưởng cho lập trình viên và các đội phát triển phần mềm muốn tối ưu hóa hiệu suất làm việc.

1.4.2. KHUYẾT ĐIỂM CỦA LINUX

- Hỗ trợ ứng dụng hạn chế: Linux không có nhiều ứng dụng như Windows.
- Thời gian thích nghi: Người dùng có thể mất thời gian để làm quen với Linux khi chuyển từ hệ điều hành khác.
- Không phù hợp với mọi người dùng: Linux không phải là lựa chọn tốt cho những người sử dụng máy tính một cách thông thường.
- Khả năng tương thích phần mềm: Các ứng dụng thiết kế đồ họa chuyên nghiệp và phần mềm văn phòng chuyên sâu không có phiên bản chính thức nên phải dùng các phần mềm thay thế mã nguồn mở.
- Phân mảnh quá lớn: Vì Linux có hàng trăm phiên bản khác nhau nên các nhà phát triển phần mềm gặp khó khăn trong việc tạo ra một tệp cài đặt chung cho tất cả.

CHƯƠNG 2. TỔNG QUAN KIẾN TRÚC CỦA LINUX KERNEL

2.1. KHÁI NIỆM VÀ CHỨC NĂNG CỦA LINUX KERNEL

- Kernel là thành phần cốt lõi, quan trọng nhất của hệ điều hành nói chung và hệ điều hành Linux nói riêng.
- Khái niệm: Linux kernel (hay nhân Linux) là thành phần lõi trung tâm, là "trái tim" hoặc "bộ não" cốt lõi của hệ điều hành Linux, hoạt động như một cầu nối tuyệt đối giữa phần cứng máy tính và các ứng dụng phần mềm.
- Linux kernel (nhân Linux) có rất nhiều chức năng quan trọng mà chúng ta có những chức năng cốt lõi, thiết yếu cho hiệu suất và độ ổn định cho tổng thể của hệ điều hành Linux. Các chức năng này bao gồm:
 - + Trừu tượng hóa và quản lý phần cứng:
 - Nhân Linux quản lý và trừu tượng hóa phần cứng bên dưới, cho phép phần mềm tương tác với hệ thống mà không cần biết các chi tiết cụ thể của nó.
 - Nhân hệ điều hành cung cấp một lớp trừu tượng giữa các thành phần phần cứng và phần mềm, cho phép các ứng dụng sử dụng tài nguyên hệ thống mà không cần giao tiếp trực tiếp với các thiết bị vật lý. Lớp trừu tượng này cho phép nhân hệ điều hành quản lý và kiểm soát quyền truy cập vào các tài nguyên phần cứng, chẳng hạn như bộ xử lý, bộ nhớ và thiết bị đầu vào/đầu ra (I/O), đảm bảo rằng mỗi thành phần hệ thống có thể sử dụng hiệu quả các tài nguyên sẵn có.
 - + Quản lý tiến trình:
 - Tiến trình là một thực thể điều khiển đoạn mã lệnh cho chương trình hay dịch vụ trong hệ thống.
 - Một chức năng quan trọng khác của nhân Linux là quản lý tiến trình. Nhân chịu trách nhiệm xử lý việc tạo, lập lịch và chấm dứt các tiến trình, cũng như quản lý và sử dụng bộ nhớ. Bằng cách thực thi sự cô lập tiến trình, nhân ngăn chặn các tiến trình can thiệp vào hoạt động của nhau, đảm bảo tính ổn định và độ tin cậy của toàn hệ thống.
 - Khả năng quản lý tiến trình của nhân hệ điều hành cũng cho phép sử dụng hiệu quả

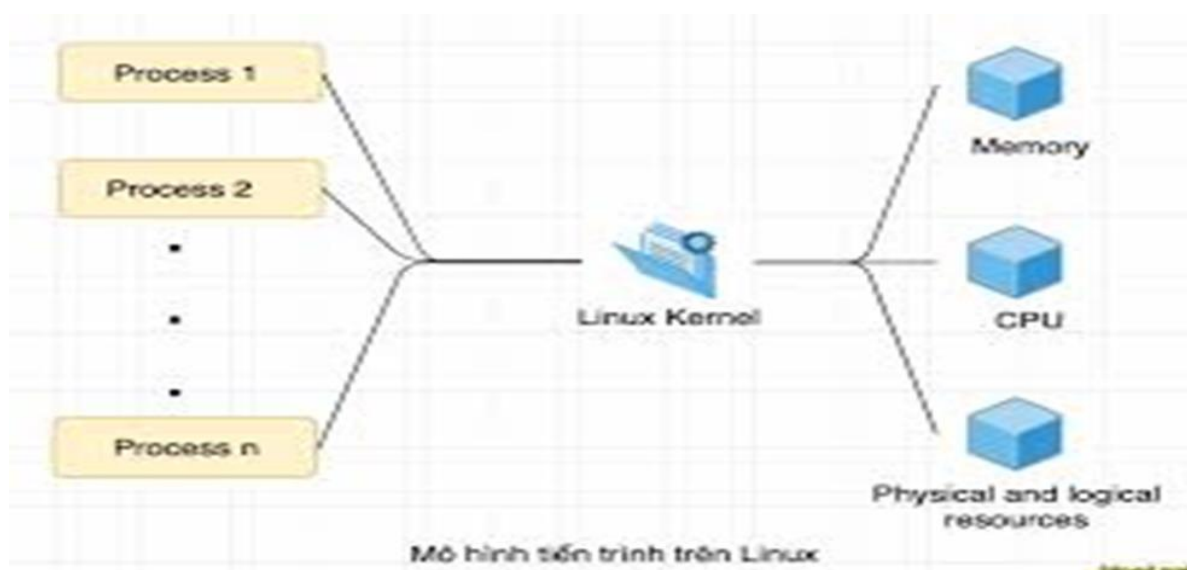
các tài nguyên hệ thống, vì nó có thể phân bổ và giải phóng không gian nhân khi cần thiết để hỗ trợ các tiến trình khác nhau đang chạy trên hệ thống.

+ Quản lý hệ thống tập tin: Nhân Linux cũng cung cấp giao diện để truy cập và quản lý các tập tin và thư mục. Nó xử lý việc lưu trữ, truy cập và cấp quyền cho các hệ thống tập tin khác nhau, cho phép các ứng dụng và người dùng tương tác với cơ sở hạ tầng lưu trữ bên dưới. Chức năng quản lý hệ thống tập tin này rất cần thiết cho việc tổ chức và duy trì dữ liệu trong hệ điều hành Linux.

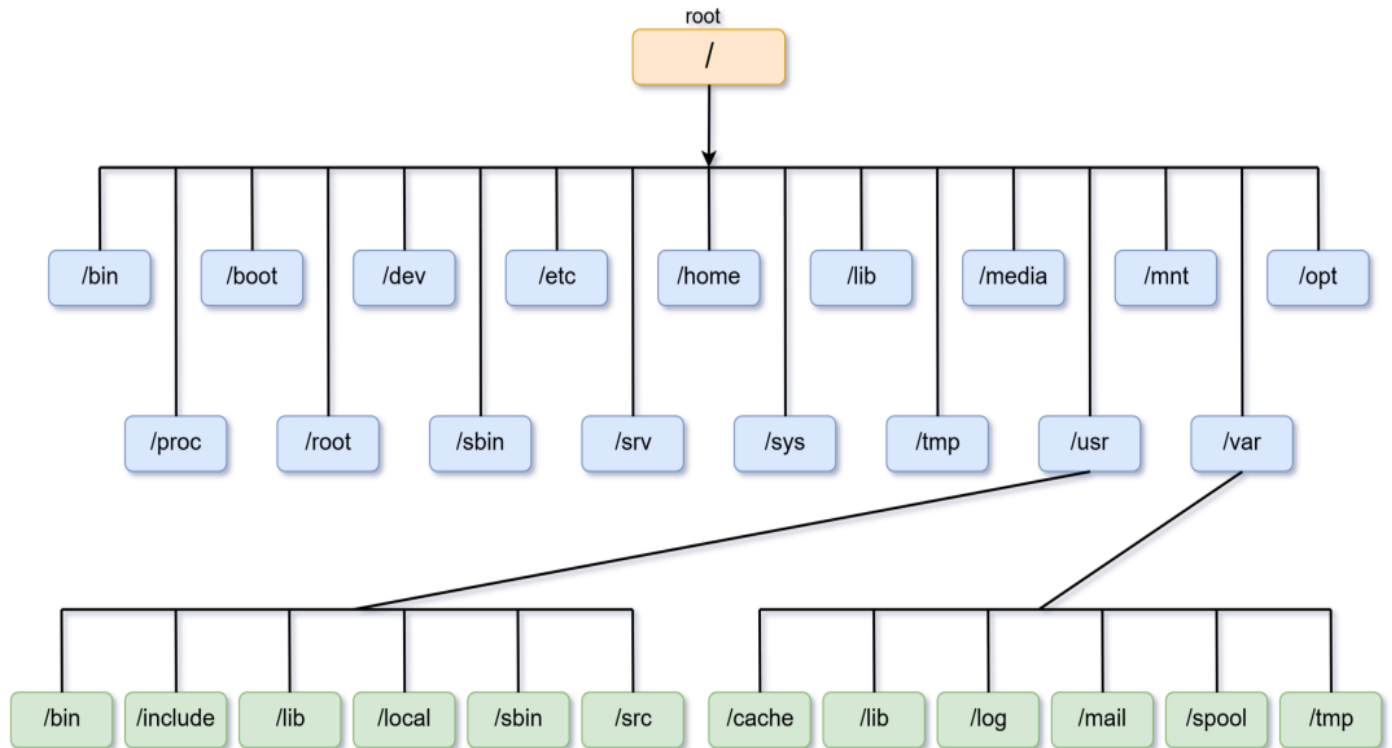
+ Quản lý mạng: Nhân Linux chịu trách nhiệm quản lý mạng, bao gồm việc triển khai các giao thức mạng như TCP/IP và quản lý các giao diện mạng và lưu lượng truy cập. Khả năng quản lý mạng này cho phép giao tiếp liền mạch giữa hệ điều hành và môi trường mạng rộng lớn hơn, cho phép các ứng dụng và dịch vụ kết nối và trao đổi dữ liệu với các hệ thống và tài nguyên từ xa.

+ Bảo mật hệ thống:

- Kiểm soát quyền người dùng và nhóm.
- Quản lý quyền truy cập tệp.
- Hỗ trợ SELinux và AppArmor.
- Ngăn chặn truy cập trái phép vào tài nguyên hệ thống.



Hình 2.1. Quản lý tiến trình của Linux Kernel



Hình 2.2. Quản lý hệ thống tập tin trong nhân Linux

2.2. LỊCH SỬ PHÁT TRIỂN CỦA LINUX KERNEL

- Nhân Linux là một nhân giống Unix miễn phí và mã nguồn mở được sử dụng trong nhiều hệ thống máy tính trên toàn thế giới.
- Kernel này được Linus Torvalds tạo ra vào năm 1991 cho máy tính cá nhân của mình và nhanh chóng được chấp nhận làm nhân cho hệ điều hành GNU (OS), được tạo ra để thay thế miễn phí cho Unix.
- Hầu hết mã hạt nhân được viết bằng C như được hỗ trợ bởi Bộ sưu tập Trình biên dịch GNU, có các phần mở rộng vượt ra ngoài C tiêu chuẩn. Mã cũng chứa mã hợp ngữ cho logic cụ thể của kiến trúc như tối ưu hóa việc sử dụng bộ nhớ và thực thi tác vụ.
- Ban đầu, cộng đồng MINIX đã đóng góp mã và ý tưởng cho nhân Linux. Vào thời điểm đó, Dự án GNU đã tạo ra nhiều thành phần cần thiết cho một hệ điều hành tự do nhưng hạt nhân riêng của nó, GNU Hurd, không đầy đủ và không có sẵn.
- Tháng 9/1991, hạt nhân Linux phiên bản 0.01 được phát hành trên máy chủ FTP của Đại

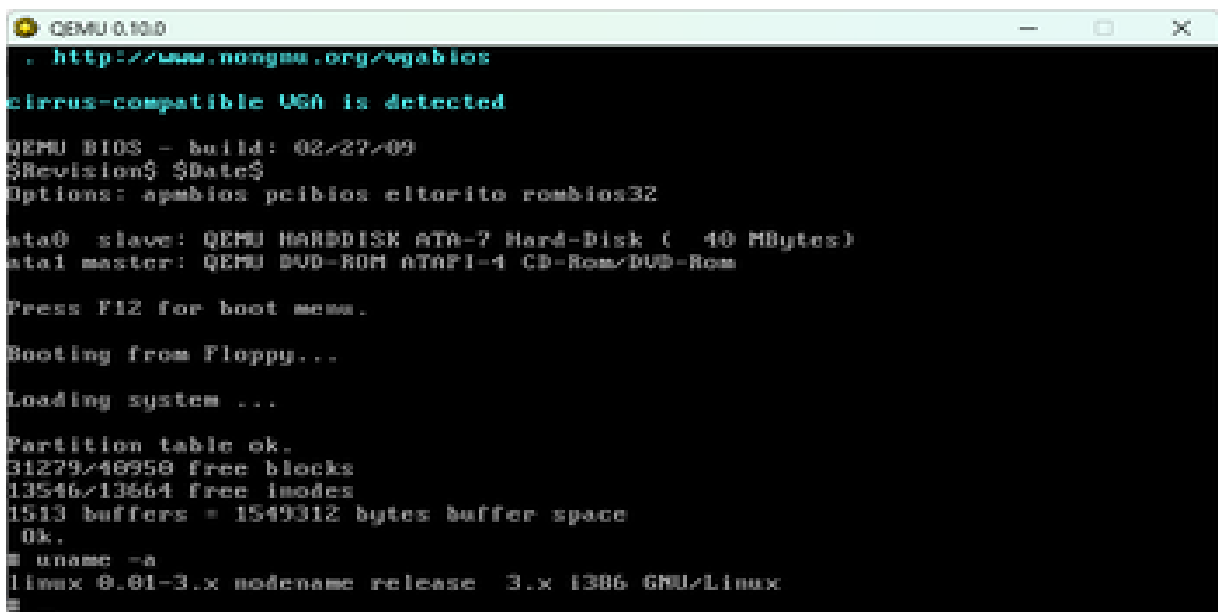
học Phần Lan và Mạng Nghiên cứu (FUNET). Tháng 10/1991, phiên bản 0.02 của hạt nhân Linux được phát hành.

- Tháng 12/1991, phiên bản 0.11 của hạt nhân Linux đã được phát hành. Đây là phiên bản đầu tiên được tự lưu trữ vì hạt nhân Linux 0.11 có thể được biên dịch bởi một máy tính chạy cùng phiên bản hạt nhân.

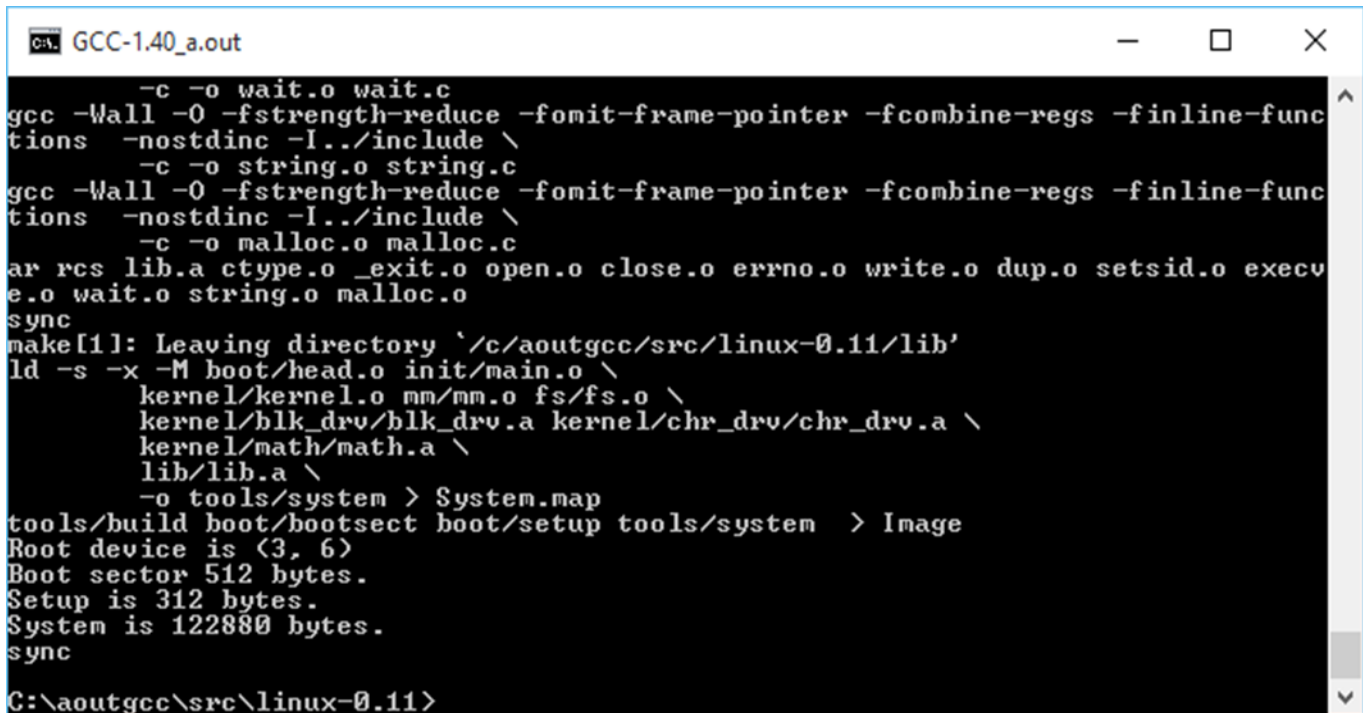
- Mã nguồn nhân Linux được bảo trì mà không cần sự trợ giúp của hệ thống quản lý mã nguồn tự động (SCM), chủ yếu là do Torvalds không thích các hệ thống SCM tập trung. Năm 2002, phát triển nhân Linux đã chuyển sang BitKeeper, một hệ thống SCM đáp ứng các yêu cầu kỹ thuật của Torvalds.

- Phiên bản 2.0.0 của Linux kernel được phát hành vào ngày 9/6/1996. Trái ngược với Unix, tất cả mã nguồn với Linux kernel có sẵn miễn phí, bao gồm trình điều khiển, thư viện runtime và các công cụ phát triển.

- Các nhà phát triển đóng góp cho nhân Linux đã nghĩ rằng điều quan trọng là hạt nhân mà Torvald đã viết cho các PC của Intel hỗ trợ các kiến trúc phần cứng khác nhau. Hiện nay hạt nhân Linux có thể chạy trên các CPU từ Intel (80386, 80486, 80686), Digital Equipment Corporation (Alpha), Motorola (MC680x0 và PowerPC), Silicon Graphics (MIPS) và Sun Microsystems (SPARC).



Hình 2.3. Linux 0.01 trong QEMU



```

GCC-1.40_a.out
-c -o wait.o wait.c
gcc -Wall -O -fstrength-reduce -fomit-frame-pointer -fcombine-regs -finline-func
tions -nostdinc -I../include \
-c -o string.o string.c
gcc -Wall -O -fstrength-reduce -fomit-frame-pointer -fcombine-regs -finline-func
tions -nostdinc -I../include \
-c -o malloc.o malloc.c
ar rcs lib.a ctype.o _exit.o open.o close.o errno.o write.o dup.o setsid.o execu
e.o wait.o string.o malloc.o
sync
make[1]: Leaving directory `/c/aoutgcc/src/linux-0.11/lib'
ld -s -x -M boot/head.o init/main.o \
    kernel/kernel.o mm/mm.o fs/fs.o \
    kernel/blk_drv/blk_drv.a kernel/chr_drv/chr_drv.a \
    kernel/math/math.a \
    lib/lib.a \
    -o tools/system > System.map
tools/build boot/bootsect boot/setup tools/system > Image
Root device is (3, 6)
Boot sector 512 bytes.
Setup is 312 bytes.
System is 122880 bytes.
sync
C:\aoutgcc\src\linux-0.11>

```

Hình 2.4. Linux 0.11

2.3. KIẾN TRÚC CỦA LINUX KERNEL

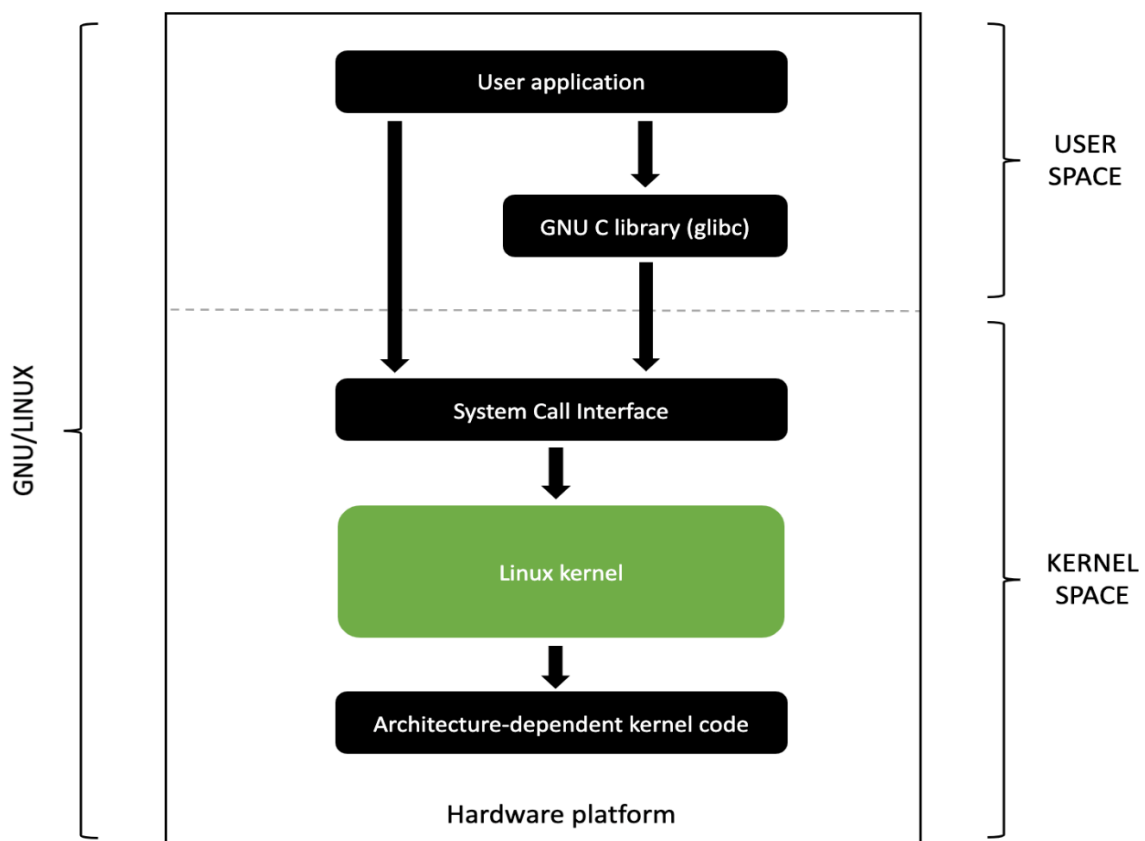
2.3.1. ĐẶC ĐIỂM

- Đối với Linux, phần kernel được xem là quan trọng nhất bởi vì nó là bộ xương sống của hệ điều hành Linux.
- Sau đây, chúng ta đi tìm hiểu về những đặc điểm trong kiến trúc của Linux kernel:
 - + Linux kernel thuộc dạng kiến trúc monolithic kernel.
 - + Có sự hỗ trợ của các Module.
 - + Đa nhiệm và đa người dùng.
 - + Có sự phân vùng không gian giữa User space và Kernel space.
 - + Giao tiếp qua System Calls.
 - + Khả năng đa nền tảng.
 - + Quản lý tài nguyên tiên tiến.

2.3.2. THÀNH PHẦN CHÍNH TRONG LINUX KERNEL

Đối với Linux Kernel, các thành phần chính bao gồm:

- User application: Chính là các ứng dụng desktop Skype, Chrome IntelliJ... hoặc là các ứng dụng phần mềm mà chúng ta viết ra.
- GNU C library (glibc): Hiểu đơn giản nó là các thư viện viết bằng C. Nó bao gồm các API tiêu chuẩn ví dụ như đọc/ghi file, xử lý chuỗi... hoặc thực hiện gọi System Call Interface xuống kernel.
- User space: Vùng không gian dành cho user, là nơi các chương trình được thực thi.
- Kernel space: Vùng không gian kernel, nơi kernel hoạt động và tương tác với hệ thống hardware.
- System Call Interface: Hai vùng user space và kernel space hoạt động độc lập trên vùng nhớ khác nhau. User space không thể truy cập trực tiếp vào Kernel space. System Call là cánh cửa bí mật đi đến kernel. Các User application cần thông qua nó để yêu cầu kernel thực thi một nhiệm vụ. Ngoài ra System call bao gồm các device files để làm việc với device drivers, nói cách khác là tương tác với các thiết bị ngoại vi (mouse, keyboard...).



Hình 2.5. Kiến trúc của Linux Kernel

2.3.3. LINUX KERNEL THUỘC KIẾN TRÚC MONOLITHIC

2.3.3.1. TÌM HIỂU CÁC LOẠI KIẾN TRÚC KERNEL

Về bản chất, có nhiều cách để xây dựng cấu trúc và biên dịch 1 bộ kernel nhất định từ đầu. Nhìn chung, với hầu hết các kernel hiện nay, chúng ta có thể chia ra làm 3 loại chính: monolithic, microkernel, hybrid và các loại kiến trúc kernel này đều có những ưu điểm và khuyết điểm đặc trưng để chúng ta nhận biết được.

- Microkernel:

+ Microkernel là lượng phần mềm gần như tối thiểu có thể cung cấp các cơ chế cần thiết để triển khai một hệ điều hành.

+ Microkernel có đầy đủ các tính năng cần thiết để quản lý bộ vi xử lý, bộ nhớ và IPC. Có rất nhiều thứ khác trong máy tính có thể được nhìn thấy, tiếp xúc và quản lý trong chế độ người dùng. Microkernel có tính linh hoạt khá cao, vì vậy bạn không phải lo lắng khi thay đổi 1 thiết bị nào đó, ví dụ như card màn hình, ổ cứng lưu trữ... hoặc thậm chí là cả hệ điều hành. Microkernel với những thông số liên quan footprint rất nhỏ, tương tự với bộ nhớ và dung lượng lưu trữ, chúng còn có tính bảo mật khá cao vì chỉ định rõ ràng những tiến trình nào hoạt động trong chế độ user mode, mà không được cấp quyền như trong chế độ giám sát - supervisor mode.

+ Ưu điểm:

- Tính linh hoạt cao.
- Bảo mật.
- Sử dụng ít footprint cài đặt và lưu trữ.

+ Khuyết điểm:

- Phần cứng đôi khi “khó hiểu” hơn thông qua hệ thống driver.
- Phần cứng hoạt động dưới mức hiệu suất thông thường vì các trình điều khiển ở trong chế độ user mode.
- Các tiến trình phải chờ đợi để được nhận thông tin.
- Các tiến trình không thể truy cập tới những ứng dụng khác mà không phải chờ đợi.

- Monolithic Kernel:

+ Monolithic Kernel là một kiến trúc hệ điều hành với toàn bộ hệ điều hành chạy trong

không gian nhân.

+ Với Monolithic thì khác, chúng có chức năng bao quát rộng hơn so với microkernel, không chỉ tham gia quản lý bộ vi xử lý, bộ nhớ, IRC, chúng còn can thiệp vào trình điều khiển driver, tính năng điều phối file hệ thống, các giao tiếp qua lại giữa server... Monolithic tốt hơn khi truy cập tới phần cứng và đa tác vụ, bởi vì nếu 1 chương trình muốn thu thập thông tin từ bộ nhớ và các tiến trình khác, chúng cần có quyền truy cập trực tiếp và không phải chờ đợi các tác vụ khác kết thúc. Nhưng đồng thời, chúng cũng là nguyên nhân gây ra sự bất ổn vì nhiều chương trình chạy trong chế độ supervisor mode hơn, chỉ cần 1 sự cố nhỏ cũng khiến cho cả hệ thống mất ổn định.

+ Ưu điểm:

- Truy cập trực tiếp đến các phần cứng.
- Dễ dàng xử lý các tín hiệu và liên lạc giữa nhiều thành phần với nhau.
- Nếu được hỗ trợ đầy đủ, hệ thống phần cứng sẽ không cần cài đặt thêm driver cũng như phần mềm khác.
- Quá trình xử lý và tương tác nhanh hơn vì không cần phải chờ đợi.

+ Khuyết điểm:

- Tiêu tốn nhiều footprint cài đặt và lưu trữ.
- Tính bảo mật kém hơn vì tất cả đều hoạt động trong chế độ giám sát - supervisor mode.

- Hybrid Kernel:

+ Hybrid là nhân hệ điều hành có kiến trúc cố gắng kết hợp các khía cạnh và lợi ích của kiến trúc nhân vi mô và nhân nguyên khối được sử dụng trong hệ điều hành.

+ Khác với 2 loại kernel trên, Hybrid có khả năng chọn lựa và quyết định những ứng dụng nào được phép chạy trong chế độ user hoặc supervisor. Thông thường, những thứ như driver và file hệ thống I/O sẽ hoạt động trong chế độ user mode trong khi IPC và các gói tín hiệu từ server được giữ lại trong chế độ supervisor. Tính năng này thực sự rất có ích vì chúng đảm bảo tính hiệu quả của hệ thống, phân phối và điều chỉnh công việc phù hợp, dễ quản lý.

+ Ưu điểm:

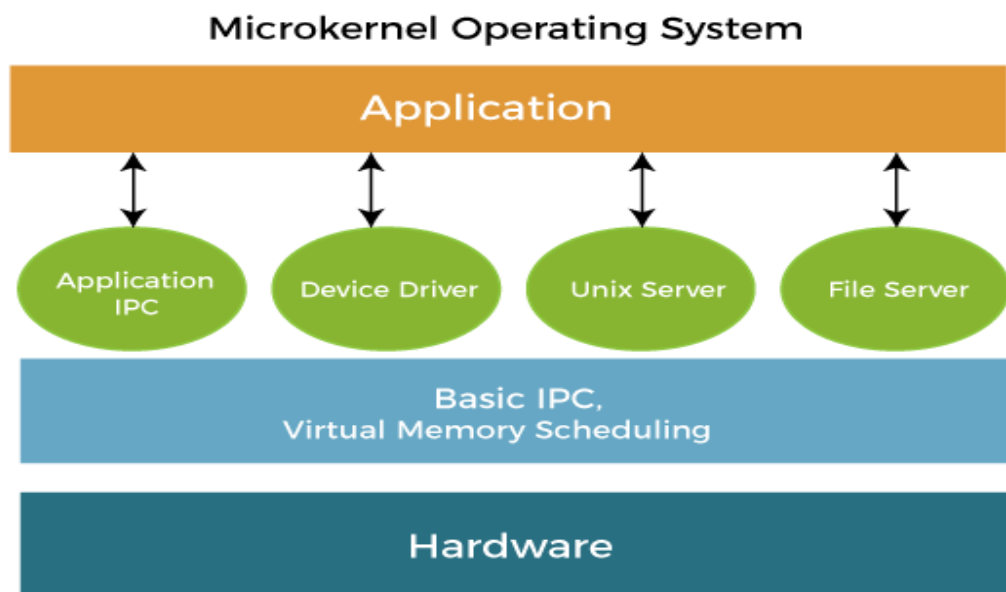
- Các nhà phát triển có thể chọn và phân loại những ứng dụng nào sẽ chạy trong chế độ

thích hợp.

- Sử dụng ít footprint hơn so với monolithic kernel.
- Có tính linh hoạt và cơ động cao nhất.

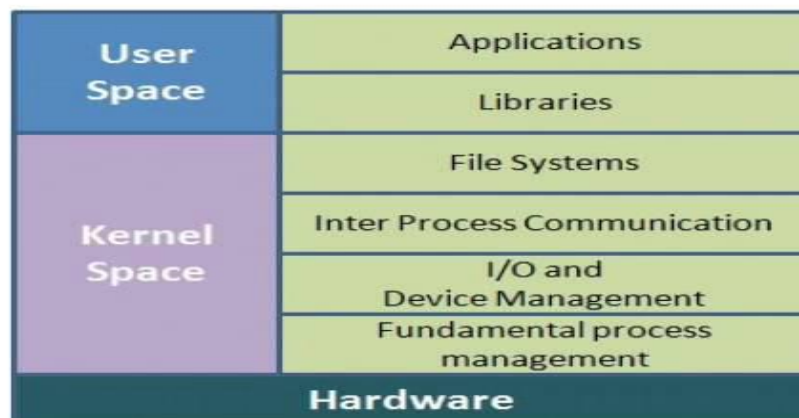
+ Khuyết điểm:

- Có thể bị bỏ lại trong quá trình gây treo hệ thống tương tự như với microkernel.
- Các trình điều khiển thiết bị phải được quản lý bởi người dùng.

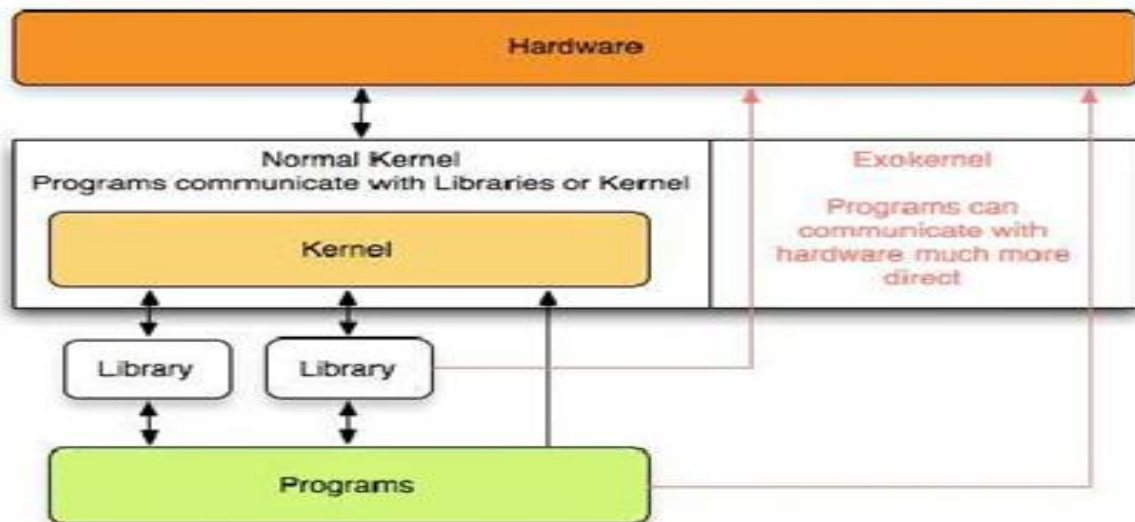


Hình 2.6. Kiến trúc Microkernel

Monolithic Kernel



Hình 2.7. Kiến trúc Monolithic



Hình 2.8. Kiến trúc Hybrid

2.3.3.2. LÝ DO LINUX KERNEL THUỘC KIẾN TRÚC MONOLITHIC

- Linux Kernel thuộc kiến trúc Monolithic chủ yếu là do hiệu năng vượt trội và lịch sử phát triển của Linux Kernel.
- Lý do:
 - + Tối đa hóa hiệu năng: Mọi dịch vụ hệ thống như quản lý bộ nhớ, tiến trình, hệ thống tệp và trình điều khiển phần cứng (driver) đều nằm chung trong một không gian địa chỉ. Điều này cho phép các thành phần giao tiếp trực tiếp với nhau bằng lệnh gọi hàm thông thường, không mất thời gian chuyển đổi ngữ cảnh hay giao tiếp giữa các tiến trình (IPC) như ở kiến trúc Microkernel.
 - + Giao tiếp phần cứng nhanh: Do driver nằm sẵn trong nhân, Linux có thể tương tác với phần cứng, đọc/ghi dữ liệu và xử lý ngắt (interrupt) với độ trễ tối thiểu.
 - + Thiết kế nguyên bản và lịch sử của nó: Vào năm 1991, kiến trúc Microkernel còn khá mới mẻ và gây tranh cãi về mặt hiệu suất. Linus Torvalds đã quyết định viết Linux theo mô hình Monolithic truyền thống để tối giản hóa thiết kế và thực thi câu lệnh nhanh nhất có thể.
- Tuy nhiên, để khắc phục khuyết điểm cồng kềnh và thiếu linh hoạt của Monolithic, Linux phát triển thêm cơ chế Modular để cho phép hệ thống nạp hoặc gỡ bỏ các driver hay tính năng ngay khi đang chạy mà không cần khởi động lại toàn bộ Kernel.

2.4. CƠ CHẾ HOẠT ĐỘNG TRONG LINUX KERNEL

2.4.1. QUÁ TRÌNH KHỞI ĐỘNG

- Quá trình khởi động (Boot Process) của Linux kernel là chuỗi tuần tự từ lúc bật nguồn (Power-on) đến khi hệ điều hành hoàn toàn sẵn sàng. Quá trình này chuyển giao quyền điều khiển qua các giai đoạn cốt lõi: Firmware (BIOS/UEFI) → Bộ nạp khởi động (Bootloader) → Linux Kernel.

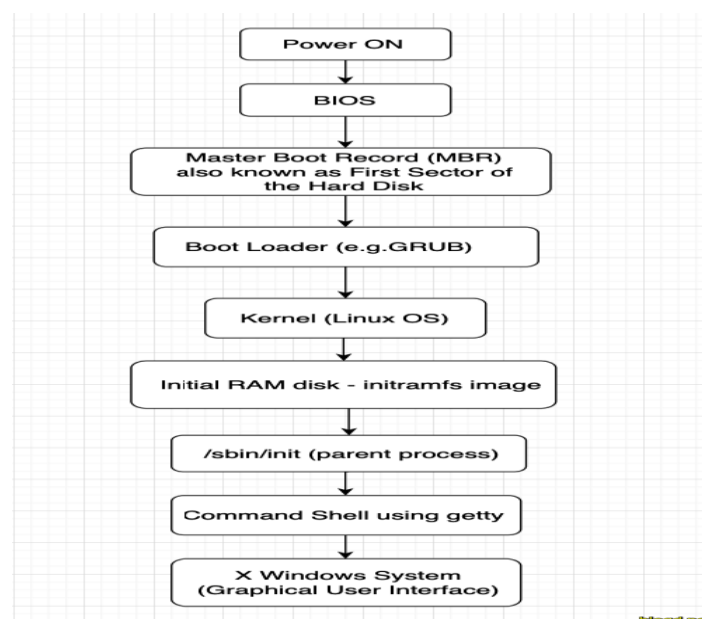
- Chi tiết các bước:

+ Firmware (BIOS / UEFI):

- Ngay khi bật nguồn, BIOS (trên máy cũ) hoặc UEFI (trên máy hiện đại) thực thi mã từ bo mạch chủ.
- Thực hiện POST (Power-On Self Test) để kiểm tra phần cứng và tìm thiết bị khởi động.
- Đọc Bootloader từ ổ đĩa dựa trên phân vùng cấu hình.

+ Bootloader:

- Hiển thị menu khởi động cho phép chọn hệ điều hành hoặc kernel.
- Nạp Kernel image và Initrd/Initramfs (hệ thống tệp tạm thời chứa các driver thiết yếu để gắn ổ đĩa chính) vào bộ nhớ RAM.
- Chuyển quyền điều khiển hệ thống sang cho kernel.



Hình 2.9. Sơ đồ các bước khởi động kernel

2.4.2. CÁCH KERNEL GIAO TIẾP VỚI PHẦN CỨNG

- Kernel giao tiếp với phần cứng thông qua các Device Driver (trình điều khiển thiết bị) và cơ chế ngắt (Interrupt), đóng vai trò là lớp trung gian dịch các yêu cầu phần mềm phức tạp thành tín hiệu điều khiển điện tử mà phần cứng hiểu được.

- Các bước:

+ Phân quyền thực thi (Execution Modes):

- User Mode: Các ứng dụng phần mềm của người dùng hoạt động ở chế độ này và không được phép truy cập trực tiếp vào phần cứng để tránh xung đột hoặc làm hỏng hệ thống.
- Kernel Mode: Kernel hoạt động ở chế độ đặc quyền cao nhất, cho phép tương tác trực tiếp với toàn bộ tài nguyên phần cứng (CPU, RAM, ổ cứng).

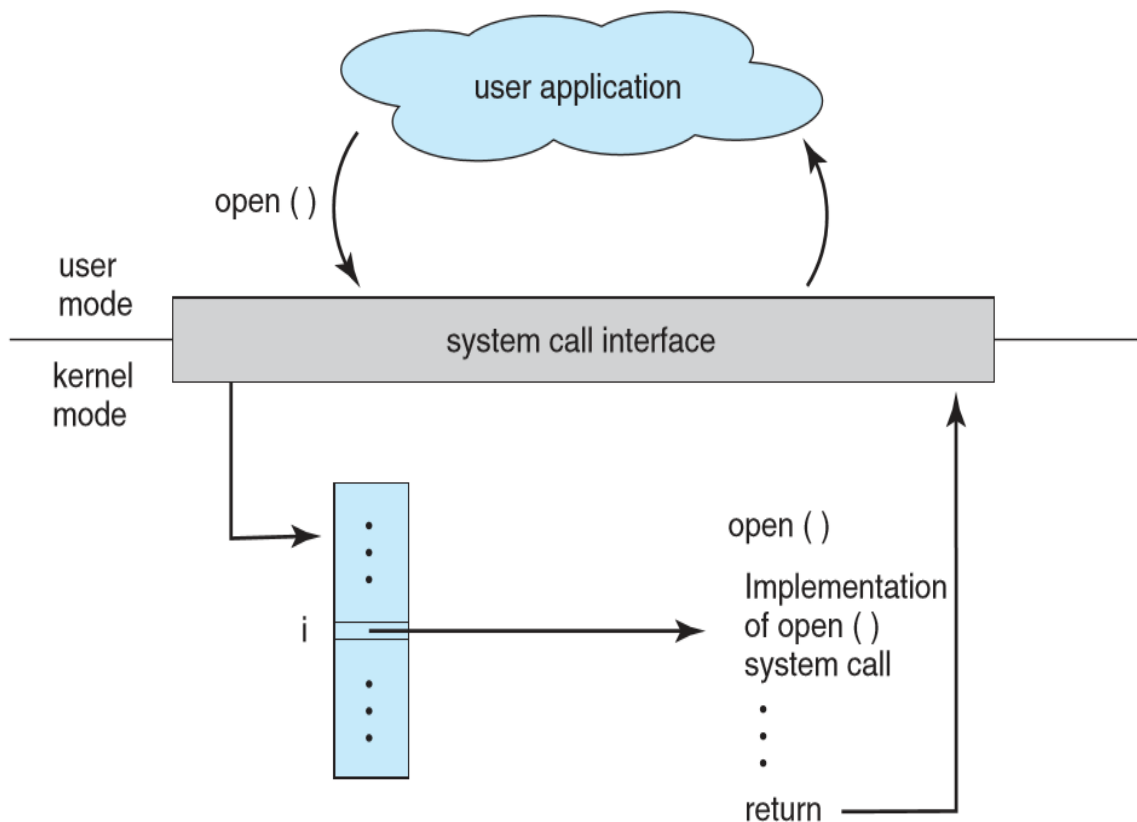
+ Các bước giao tiếp cụ thể:

- Yêu cầu (System Call): Khi ứng dụng muốn đọc file, phát âm thanh hay in tài liệu, nó gửi một System Call (lời gọi hệ thống) tới Kernel.
- Xử lý qua Driver: Kernel nhận yêu cầu, tìm đến Device Driver tương ứng của thiết bị. Driver hoạt động như một "phiên dịch viên" cụ thể cho từng loại phần cứng, chuyển đổi yêu cầu chung của hệ điều hành thành lệnh ngôn ngữ máy (Assembly/Byte code) cho thiết bị.
- Thực thi phần cứng: CPU gửi các tín hiệu điều khiển này tới thiết bị qua các bus hệ thống.
- Ngắt (Interrupt): Khi phần cứng hoàn thành tác vụ (như quét xong tài liệu hoặc nhận dữ liệu mạng), nó gửi một tín hiệu Hardware Interrupt đến CPU. Kernel tạm dừng công việc hiện tại, thực thi đoạn mã xử lý ngắt và trả dữ liệu về cho ứng dụng.

2.4.3. CƠ CHẾ SYSTEM CALL

- System call là cầu nối bắt buộc cho phép các chương trình ứng dụng (user space) yêu cầu hệ điều hành (kernel space) thực hiện các tác vụ đặc quyền như truy cập ổ đĩa, mở mạng, hoặc cấp phát bộ nhớ.

- Phân loại thường được chia thành 5 nhóm chính:
 - + Quản lý tiến trình: Tạo, kết thúc tiến trình.
 - + Quản lý tệp (File): Đọc, ghi, mở, đóng tệp.
 - + Quản lý thiết bị: Đọc/ghi thiết bị, cấu hình thiết bị.
 - + Thông tin hệ thống: Lấy thời gian, cấu hình hệ thống.
 - + Giao tiếp (Communication): Tạo kết nối, gửi/nhận dữ liệu mạng.



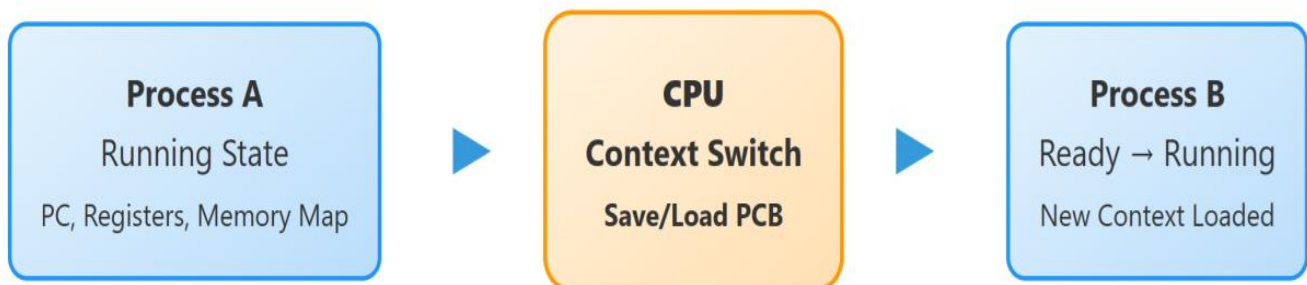
Hình 2.10. Cơ chế System Call

2.4.4. CONTEXT SWITCHING

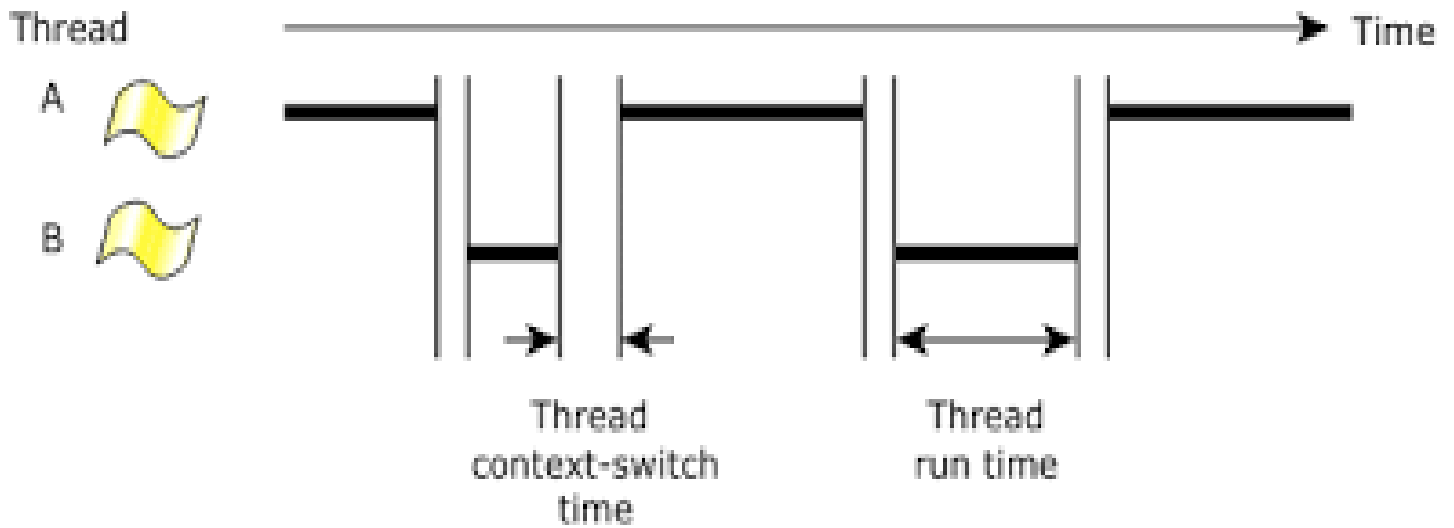
- Context switching (chuyển đổi ngữ cảnh) là quá trình Linux kernel lưu lại trạng thái (thanh ghi, stack, bộ đệm) của tiến trình/luồng đang chạy, và khôi phục trạng thái của tiến trình tiếp theo.
- Quá trình diễn ra:

- + Lưu ngữ cảnh: Lưu con trỏ ngăn xếp (stack pointer), bộ đếm chương trình (program counter), và các thanh ghi của tiến trình cũ vào Process Control Block (PCB).
 - + Chuyển đổi bộ nhớ ảo: Cập nhật bảng trang (Page Table) và chuyển đổi không gian địa chỉ bộ nhớ nếu hai tiến trình không cùng thuộc một tiến trình cha (process).
 - + Khôi phục ngữ cảnh: Tải các thanh ghi và dữ liệu của tiến trình mới từ PCB của nó vào CPU và bắt đầu thực thi.
- Các loại Context Switching:
- + Process Context Switch.
 - + Thread Context Switch.
 - + Mode Switch.
-

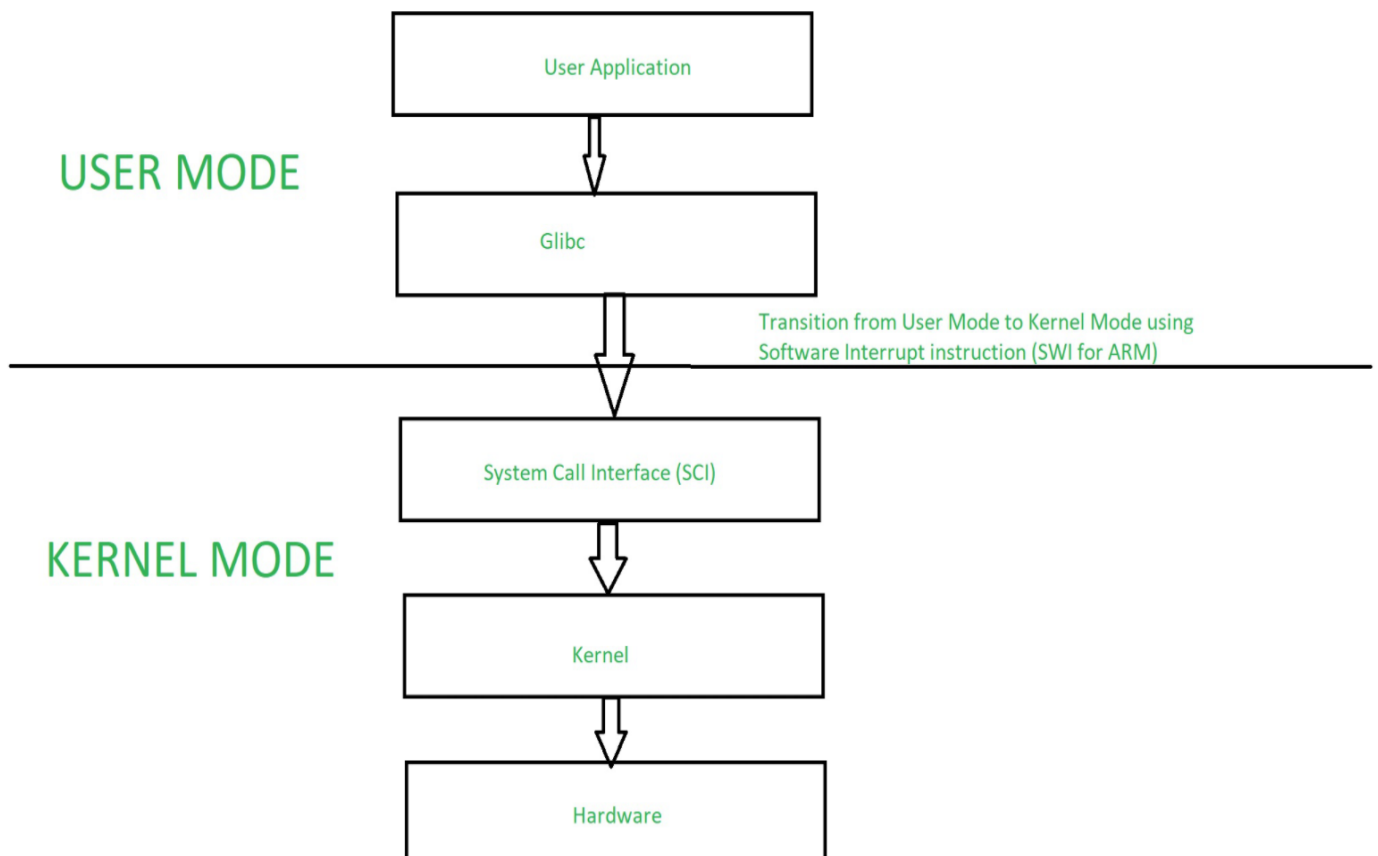
Process Context Switch Diagram



Hình 2.11. Process Context Switch



Hình 2.12. Thread Context Switch



Hình 2.13. Mode Switch

2.5. ƯU ĐIỂM VÀ KHUYẾT ĐIỂM CỦA LINUX KERNEL

2.5.1. ƯU ĐIỂM CỦA LINUX KERNEL

- Mã nguồn mở: Cho phép mọi người tự do xem, sửa đổi và phân phối lại. Điều này thúc đẩy cộng đồng toàn cầu liên tục vá lỗi và nâng cấp.
- Tính linh hoạt & Tùy biến cao: Có thể tinh chỉnh, biên dịch hoặc thêm/bớt các mô-đun để phù hợp với mọi thiết bị, từ các hệ thống siêu máy tính lớn đến các thiết bị IoT nhỏ gọn.
- Khả năng tương thích phần cứng: Hỗ trợ đa dạng các kiến trúc phần cứng khác nhau (x86, ARM, RISC-V,...), giúp nó chạy mượt mà trên nhiều thiết bị.
- Độ ổn định cực cao: Rất hiếm khi xảy ra tình trạng sập hệ thống. Máy chủ chạy Linux có thể hoạt động liên tục trong nhiều năm mà không cần khởi động lại.

2.5.2. KHUYẾT ĐIỂM CỦA LINUX KERNEL

- Trước hết là do Linux kernel thuộc kiến trúc Monolithic nên sẽ gặp phải tình trạng có lỗi cao và kích thước công kênh:
 - + Nguy cơ lỗi cao: Linux sử dụng nhân nguyên khối, nghĩa là tất cả các dịch vụ hệ thống cốt lõi đều chạy trong cùng một không gian bộ nhớ.
 - + Kích thước công kênh: Vì tập hợp quá nhiều tính năng và driver sẵn trong nhân, kích thước nhân mặc định khá lớn và tiêu tốn nhiều bộ nhớ RAM cho các thành phần không cần thiết.
- Thứ hai, đó chính là sự bất ổn định khi thay đổi mô – đun: Linux cho phép tải và dỡ bỏ các đoạn mã chương trình vào nhân trong lúc đang chạy (Kernel Modules). Tuy nhiên, điều này phá vỡ tính ổn định nguyên khối vì các đoạn mã từ bên thứ ba (hoặc đang thử nghiệm) có thể làm hỏng các luồng khác.
- Thứ ba, vấn đề tương thích và thiếu driver:
 - + Phụ thuộc phần cứng: Vì các nhà sản xuất phần cứng thường ưu tiên viết driver cho Windows hoặc macOS, người dùng Linux thường xuyên gặp tình trạng thiếu driver chính chủ.
 - + Nếu phần cứng của bạn không được nhân Linux hỗ trợ sẵn, việc biên dịch (compile) lại kernel hoặc vá lỗi để thiết bị hoạt động đòi hỏi kiến thức chuyên môn khá cao.

CHƯƠNG 3. BẢO MẬT TRONG LINUX KERNEL

3.1. KHÁI NIỆM VÀ CHỨC NĂNG VỀ BẢO MẬT TRONG LINUX KERNEL

3.1.1. KHÁI NIỆM VỀ BẢO MẬT TRONG LINUX KERNEL

- Bảo mật trong Linux Kernel là lớp phòng thủ cốt lõi của hệ điều hành, quản lý sự phân tách giữa user-space và kernel-space để áp dụng và tập hợp các cơ chế bảo vệ ở tầng lõi hệ điều hành nhằm ngăn chặn truy cập trái phép, cô lập tiến trình độc hại và giới hạn độc quyền.

3.1.2. CHỨC NĂNG VỀ BẢO MẬT TRONG LINUX KERNEL

- Phân quyền và kiểm soát truy cập (MAC): Sử dụng các mô – đun như SELinux hoặc AppArmor để giới hạn chặt chẽ quyền hành động của từng tiến trình và ứng dụng.
- Secure Computing: Lọc các lời gọi hệ thống mà một tiến trình được phép sử dụng.
- Bảo mật phần cứng/ bộ nhớ: Cung cấp các công nghệ như ASLR (Address Space Layout Randomization), NX/XD bit và KASLR (Kernel Address Space Layout Randomization) để chống lại các cuộc tấn công tràn bộ đệm hoặc thực thi mã độc.
- Kernel Lockdown: Tùy chọn cấu hình ngăn chặn tài khoản root sửa đổi trực tiếp mã nhân đang chạy, tránh việc chen mã độc vào hệ thống.
- Linux Capabilities: Chia nhỏ quyền của tài khoản quản trị root thành các quyền độc lập, ngăn chặn việc một tiến trình bị xâm nhập chiếm toàn bộ hệ thống.

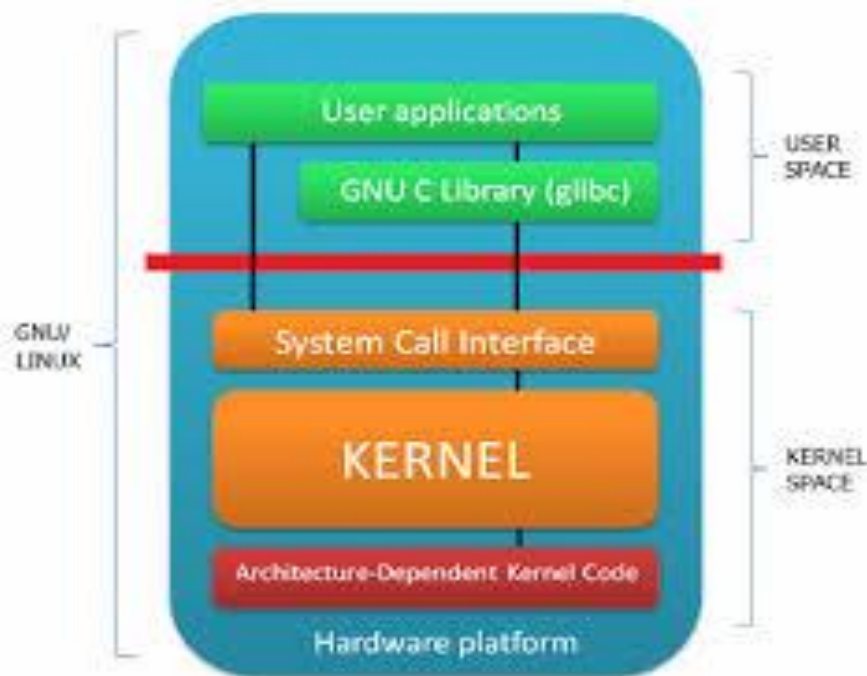


Hình 3.1. Chức năng phân quyền và kiểm soát truy cập trong bảo mật Linux Kernel

3.2. MỘT SỐ NGUYÊN TẮC BẢO MẬT TRONG LINUX KERNEL

3.2.1. ĐẶC QUYỀN TỐI THIỂU

- Phân chia rõ ràng giữa không gian người dùng (User Space) và không gian hạt nhân (Kernel Space).
- User space và kernel space là hai vùng nhớ được hệ điều hành (như Linux, Windows) chia tách độc lập. User space là nơi chạy các ứng dụng của bạn (trình duyệt, phần mềm), trong khi kernel space là vùng lõi có đặc quyền cao nhất để quản lý phần cứng và tài nguyên hệ thống.
- Mọi tiến trình chạy ở User Space đều không được phép can thiệp trực tiếp vào bộ nhớ hoặc phần cứng nếu không có lời gọi hệ thống (system call) được cấp phép.



Hình 3.2. Nguyên tắc đặc quyền tối thiểu

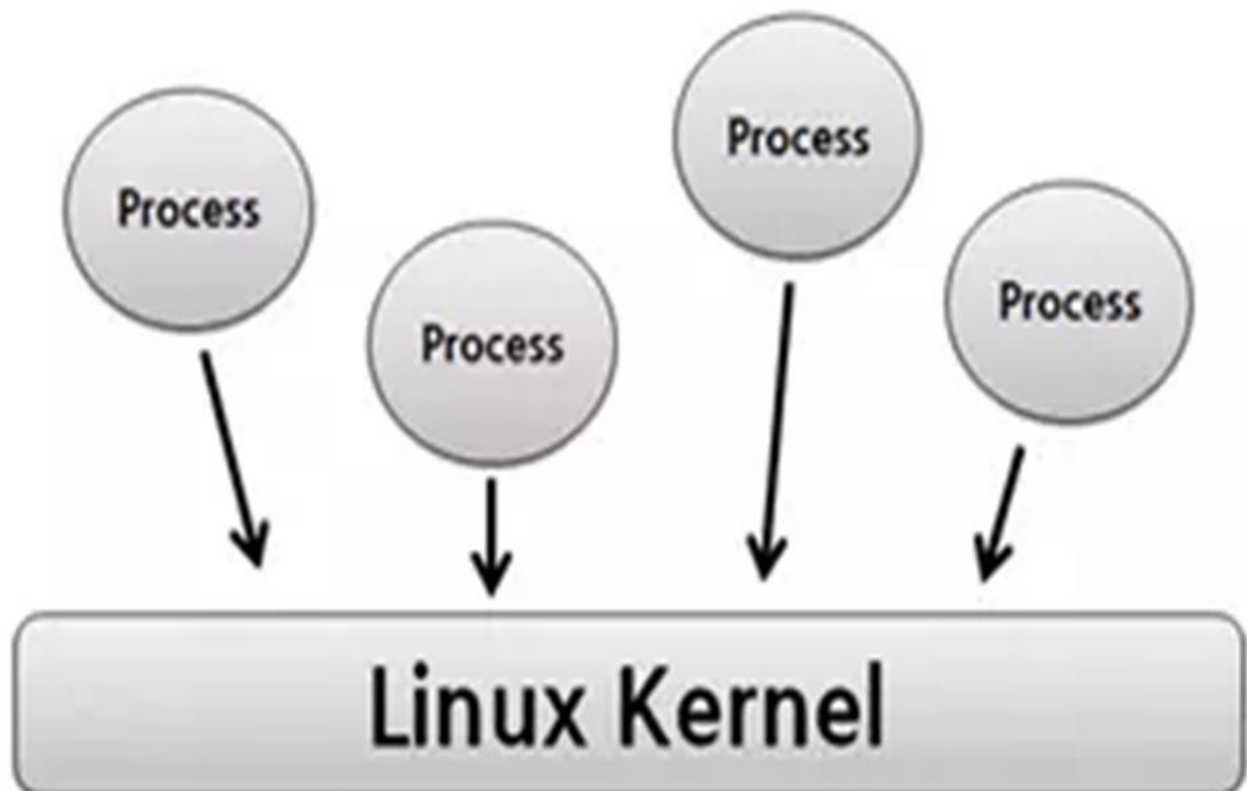
3.2.2. CẬP NHẬT VÀ BẮN VÁ

- Thường xuyên cập nhật bản vá bảo mật. Các giải pháp như Livepatching (VD: Canonical Livepatch, Kpatch) cho phép vá lỗ hổng Kernel ngay trong lúc hệ thống đang chạy mà không cần khởi động lại.
- Linux Kernel thường xuyên:
 - + Vá lỗ hổng bảo mật.

- + Cập nhật driver.
- + Sửa lỗi privilege escalation.
- Quản trị viên (admin) cần:
- + Update kernel định kỳ.
- + Không dùng kernel quá cũ.

3.2.3. CÁCH LY TIỀN TRÌNH

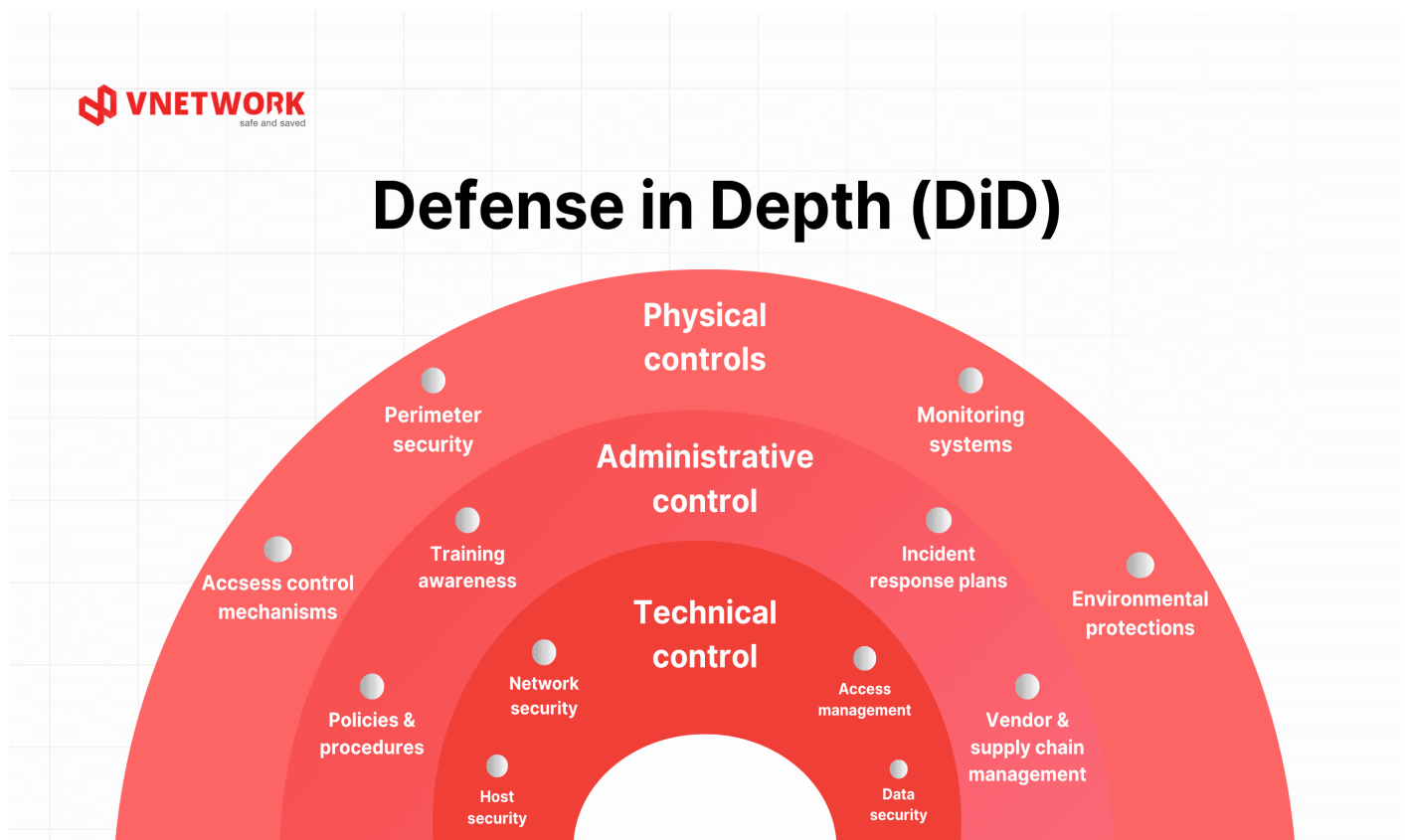
- Mỗi tiến trình chạy trong không gian bộ nhớ riêng để:
- + Không đọc dữ liệu của tiến trình khác.
- + Không ghi đè vùng nhớ của hệ thống.
- Kernel dùng: Virtual Memory, User Space và Kernel Space.
- Nếu tiến trình lỗi: Không làm sập toàn bộ hệ điều hành là bởi vì nhờ cơ chế phân vùng bộ nhớ và đặc quyền, điển hình là sự phân chia giữa không gian người dùng (User Space) và không gian nhân (Kernel Space).



Hình 3.3. Cách ly tiến trình

3.2.4. PHÒNG THỦ NHIỀU LỚP

- Phòng thủ nhiều lớp trong Linux kernel (Defense-in-Depth) là chiến lược sử dụng các tầng bảo mật chồng chéo nhau bên trong nhân hệ điều hành.
- Linux Kernel không dựa vào một cơ chế bảo mật duy nhất.
- Nó kết hợp:
 - + Firewall.
 - + File permission.
 - + SELinux/AppArmor.
 - + Audit log.
 - + Encryption.
 - + Secure Boot.
- Nếu một lớp bị vượt qua: Các lớp khác vẫn bảo vệ hệ thống.



Hình 3.4. Phòng thủ nhiều lớp trong Linux Kernel

3.3. MỘT SỐ CÁCH BẢO MẬT TRONG LINUX KERNEL

3.3.1. CHẶN TRUY CẬP BẰNG FIREWALL

- Một trong những cách bảo mật Linux đơn giản nhất là kích hoạt và cấu hình firewall. Firewall hoạt động như một hàng rào giữa lưu lượng truy cập chung của internet và máy tính của mình. Cụ thể, các firewall này sẽ xem xét cẩn thận các lưu lượng vào và ra. Từ đó quyết định có nên cho phép gửi thông tin hay không.
- Firewall sẽ kiểm tra các lưu lượng dựa trên một bộ quy tắc (rule) được cấu hình sẵn bởi người dùng. Thông thường, một server sẽ chỉ sử dụng một vài port mạng cho các dịch vụ hợp pháp. Các cổng còn lại không được sử dụng, và sẽ được bảo vệ an toàn ở sau firewall. Và firewall sẽ chặn mọi lưu lượng đi đến các cổng này.
- Do đó, ta có thể dễ dàng loại bỏ dữ liệu không mong muốn. Thậm chí là cấu hình các điều kiện cho việc sử dụng các dịch vụ. Hiện nay có khá nhiều giải pháp firewall có sẵn để bảo mật Linux.



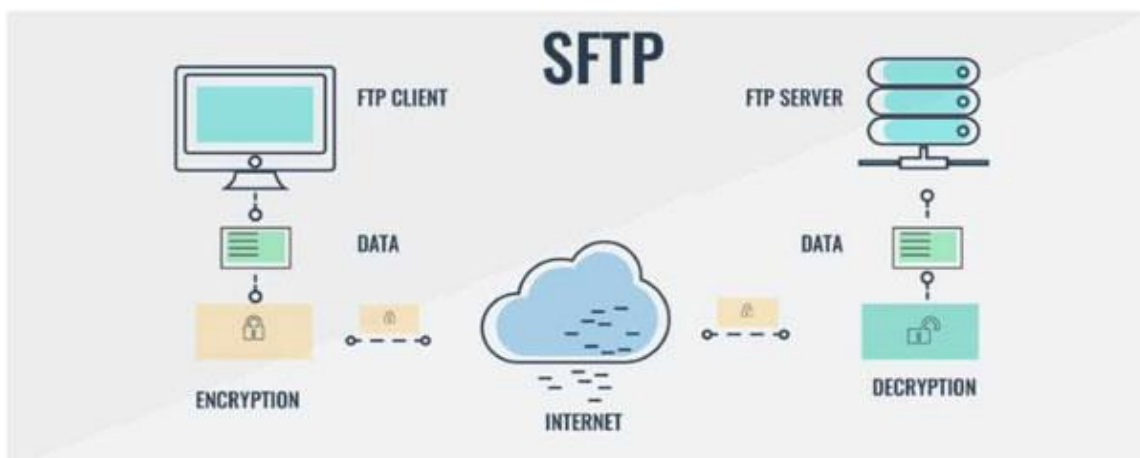
Hình 3.5. Bảo mật Linux bằng cách chặn truy cập bằng Firewall

3.3.2. CHÚ Ý CẬP NHẬT THƯỜNG XUYỀN

Các nhà bảo trì phân phối luôn cố gắng cập nhật các bản vá mới nhất để khắc phục sớm mọi lỗ hổng phần mềm trên hệ thống. Do đó, hãy luôn chú ý đến các bản cập nhật và thường xuyên cài đặt chúng để bảo vệ hệ thống Linux của mình.

3.3.3. SỬ DỤNG SFTP THAY VÌ FTP

- SFTP (viết tắt của Secure File Transfer Protocol) là một giao thức mạng cho phép bạn truyền tải, truy cập và quản lý tệp tin an toàn giữa máy tính cá nhân và máy chủ từ xa.
- FTP (File Transfer Protocol) là Giao thức truyền tải tệp tin trên mạng máy tính. Chức năng chính của FTP là cho phép người dùng tải các tệp tin (hình ảnh, tài liệu, code,...) từ máy tính cá nhân lên máy chủ (upload) hoặc tải tệp tin từ máy chủ về máy tính cá nhân (download).
- Đây là việc không hề dễ dàng cho nhiều người nhưng ta cần phải biết, FTP vốn dĩ là một giao thức không an toàn. Tất cả xác thực đều được gửi dưới dạng text thuần túy. Do đó, bất kỳ ai theo dõi kết nối giữa server và máy cục bộ để có được thông tin đăng nhập của ta.
- Thực tế, có rất ít trường hợp mà ta cần triển khai FTP thì chỉ cần sử dụng FTP là đủ. Ngoài ra, FTP cũng có thể được dùng khi cần chuyển các file giữa hai máy tính. Và hai máy này được đặt sau một firewall có NAT. Lúc đó kết nối hoàn toàn được bảo mật.
- Còn lại, chúng ta nên sử dụng một giải pháp thay thế an toàn hơn như vậy để bảo mật Linux. Bộ SSH có một giao thức thay thế gọi là SFTP, hoạt động tương tự như FTP. Nhưng nó lại có được tính bảo mật tuyệt vời của giao thức SSH.
- Khi đó, ta có thể an tâm chuyển các thông tin đến và đi từ server của mình. Bên cạnh đó, hầu hết các FTP client cũng có thể giao tiếp với server SFTP.



Hình 3.6. Cách dùng SFTP thay FTP trong bảo mật Linux Kernel

CHƯƠNG 4. ỨNG DỤNG THỰC TẾ CỦA LINUX KERNEL

4.1. XU HƯỚNG PHÁT TRIỂN CỦA LINUX KERNEL

Xu hướng phát triển của Linux Kernel đang tập trung mạnh vào 3 xu hướng cốt lõi:

- Tích hợp ngôn ngữ Rust:
 - + Rust là ngôn ngữ lập trình hệ thống mã nguồn mở tập trung vào tốc độ cao và an toàn bộ nhớ được phát triển bởi Mozilla Research và hiện do Rust Foundation quản lý.
 - + Việc áp dụng Rust không còn mang tính thử nghiệm mà đã được chấp thuận áp dụng lâu dài trong nhân Linux. Rust giúp giải quyết bài toán muôn thuở về lỗi hỏng bảo mật liên quan đến quản lý bộ nhớ của ngôn ngữ C.
- Sự bùng nổ của eBPF (Extended Berkeley Packet Filter): eBPF hiện được ví như "hệ thần kinh" của Linux. Công nghệ này cho phép bạn chạy các chương trình an toàn, được kiểm duyệt trực tiếp trong không gian nhân (kernel space) mà không cần phải viết các module kernel truyền thống hoặc khởi động lại hệ thống:
 - + Giám sát: Cung cấp khả năng quan sát sâu (deep telemetry) các tiến trình, mạng và bộ nhớ với mức tiêu thụ tài nguyên cực thấp.
 - + Bảo mật & Tối ưu: Giúp các tác vụ như tường lửa, tối ưu hóa lưu lượng mạng và phát hiện phần mềm độc hại hoạt động linh hoạt, hiệu quả hơn.
- Tối ưu hóa AI, Cloud Native và tăng cường phần cứng: Với việc hệ điều hành Linux chiếm hơn 96% thị phần khối lượng công việc trên nền tảng đám mây, nhân Linux đang không ngừng phát triển để đáp ứng các yêu cầu điện toán hiện đại:
 - + Hỗ trợ phần cứng chuyên dụng: Tích hợp sâu các trình điều khiển (drivers) để tối ưu hóa khả năng tăng tốc phần cứng cho AI, máy học (Machine Learning) và kiến trúc đa lõi.
 - + Cải thiện hệ thống tệp tin và mạng: Nâng cấp các thư viện cốt lõi để tăng thông lượng và độ trễ, phục vụ cho các cụm lưu trữ lớn và mạng tốc độ cao.
- Nhờ tính linh hoạt, bảo mật và hiệu suất cao, nó có mặt ở hầu hết mọi khía cạnh của công nghệ hiện đại mà trong số đó phải kể đến như Server (máy chủ), Embedded system (Hệ thống nhúng), IoT (Internet vạn vật) và Cloud Computing (Điện toán đám mây).



Hình 4.1. Ngôn ngữ Rust

4.2. LINUX KERNEL TRONG SERVER

- Trong server (máy chủ), Linux Kernel hoạt động như chiếc “cầu nối” trực tiếp quản lý toàn bộ tài nguyên phần cứng (CPU, RAM, ổ cứng, mạng) và phân bổ chúng cho các ứng dụng hoặc dịch vụ đang chạy.
- Linux Kernel trong server có những vai trò rất cốt lõi:
 - + Quản lý CPU & tiến trình: Đảm bảo hàng ngàn yêu cầu từ người dùng (website, database, API) được xử lý đồng thời, chia nhỏ thời gian CPU và tối ưu hiệu suất đa luồng.
 - + Quản lý bộ nhớ: Cấp phát, theo dõi và thu hồi RAM để đảm bảo các ứng dụng chạy mượt mà, đồng thời sử dụng bộ nhớ ảo (swap) khi cần thiết.
 - + Hệ thống tệp: Hỗ trợ đa dạng định dạng như ext4, XFS, Btrfs để máy chủ đọc/ghi dữ liệu với tốc độ cao và độ an toàn ổn định.
 - + Xử lý mạng: Đây là thành phần cực kỳ quan trọng đối với server để điều hướng, định tuyến lưu lượng, tường lửa và xử lý hàng triệu gói tin (packet) mỗi giây mà không bị gián đoạn.
 - + Quản lý driver: Cung cấp trình điều khiển để kernel tương tác trực tiếp với các linh kiện phần cứng (card mạng, ổ NVMe, ổ HDD).
- Linux Kernel được xem là tiêu chuẩn vàng dành cho máy chủ vì những lí do sau đây:
 - + Bảo mật và độ tin cậy: Cho phép máy chủ hoạt động liên tục (uptime) suốt nhiều năm không cần khởi động lại. Cập nhật nhân (như cơ chế Livepatch) cho phép vá lỗi bảo mật

ngay lập tức.

+ Hỗ trợ container hóa: Kernel hỗ trợ các tính năng như cgroups (kiểm soát tài nguyên) và namespaces (cô lập tiến trình), tạo nền tảng vững chắc cho Docker, Kubernetes hoạt động hiệu quả.

+ Hiệu quả I/O vượt trội: Kiến trúc xử lý I/O hiện đại giúp tăng tốc độ đọc/ghi dữ liệu, giảm thiểu độ trễ cho các ứng dụng web và cơ sở dữ liệu lớn.

- Kernel Linux là lõi hệ điều hành, nhưng để vận hành máy chủ, người ta thường dùng các bản phân phối Linux đi kèm các công cụ quản lý:

+ Ubuntu Server: Phổ biến nhất, dễ sử dụng, tài liệu phong phú.

+ Debian: Nổi tiếng về tính ổn định cực cao.

+ AlmaLinux / Rocky Linux: Hệ điều hành thay thế hoàn hảo cho CentOS, tối ưu cho môi trường doanh nghiệp.

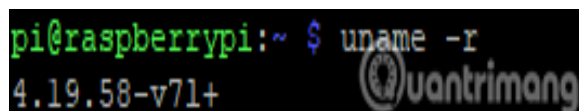
- Các lệnh quản lý Kernel cơ bản trên máy chủ:

+ Xem phiên bản Kernel hiện tại: `uname -r`.

+ Hiển thị thêm thông tin phiên bản Linux Kernel: `uname -a`.

+ Kiểm tra thông tin phần cứng hệ thống: `lshw` hoặc `lscpu`.

+ Cập nhật hệ thống và Kernel: `sudo apt update && sudo apt upgrade` (dành cho hệ máy Debian/Ubuntu) hoặc `sudo dnf update` (dành cho AlmaLinux hoặc Rocky Linux).



```
pi@raspberrypi:~ $ uname -r
4.19.58-v71+
```

Hình 4.2. Câu lệnh để kiểm tra phiên bản Kernel trong Server



```
pi@raspberrypi:~ $ uname -a
Linux raspberrypi 4.19.58-v71+ #1245 SMP Fri Jul 12 17:51:45 BST 2019 armv7l GNU
/Linux
```

Hình 4.3. Câu lệnh hiển thị thông tin phiên bản Linux Kernel

- Ưu điểm và khuyết điểm đối với linux kernel trong server:

+ Ưu điểm:

- Hiệu suất vượt trội: Kiến trúc nguyên khối (Monolithic) cho phép giao tiếp trực tiếp giữa phần cứng và phần mềm, đạt thông lượng cao và độ trễ thấp tối ưu cho máy chủ.
- Ổn định cao: Khả năng quản lý tài nguyên và đa nhiệm xuất sắc, giúp server chạy liên tục (uptime) nhiều năm liền mà không cần khởi động lại.
- Tối ưu hóa không giới hạn: Linh hoạt trong việc cấu hình. Người quản trị có thể biên dịch lại kernel, tùy chỉnh cắt bỏ các module thừa để tối ưu hoàn toàn cho một tác vụ cụ thể.
- Hỗ trợ mạng mạnh mẽ: Cung cấp sẵn các tính năng mạng cao cấp và khả năng xử lý lượng lớn kết nối đồng thời.
- Tiết kiệm chi phí: Là phần mềm mã nguồn mở miễn phí, sở hữu cộng đồng hỗ trợ lớn trên toàn cầu.

+ Khuyết điểm:

- Kiến trúc nguyên khối (Monolithic): Toàn bộ hệ thống có thể gặp lỗi (crash) dẫn đến sập toàn bộ kernel nếu một module hoặc driver bị lỗi.
- Tương thích phần cứng: Một số phần cứng mới, chuyên dụng hoặc độc quyền đôi khi không có driver chính thức ngay lập tức cho Linux.
- Độ phức tạp cao: Yêu cầu người quản trị có trình độ chuyên môn kỹ thuật cao để cấu hình, tối ưu và khắc phục sự cố sâu trong hệ thống.

- Ứng dụng: Trong các máy chủ (server), Linux Kernel có vai trò sống còn trong việc vận hành các hệ thống web quy mô lớn, điện toán đám mây, ảo hóa và mạng.

4.3. LINUX KERNEL TRONG EMBEDDED SYSTEM

- Embedded System (Hệ thống nhúng) là sự kết hợp giữa phần cứng và phần mềm máy tính, được thiết kế chuyên dụng để thực hiện các chức năng cụ thể trong một hệ thống lớn hơn.
- Linux Kernel trong Embedded System đóng vai trò là phần lõi của hệ điều hành, chịu trách nhiệm quản lý tài nguyên phần cứng, bộ nhớ, vi xử lý và giao tiếp với các thiết bị ngoại vi.

- Linux Kernel được ưa chuộng trong Embedded System với lý do:
 - + Đa dạng phần cứng: Hỗ trợ hầu hết các kiến trúc vi xử lý (như ARM, MIPS, x86, RISC-V).
 - + Mã nguồn mở: Cho phép kỹ sư tùy chỉnh (custom), tối ưu hóa kích thước và thêm/bớt tính năng để phù hợp với giới hạn tài nguyên của thiết bị.
 - + Hệ sinh thái phong phú: Cung cấp sẵn các giao thức mạng, hệ thống tệp tin (filesystem) và hàng ngàn trình điều khiển thiết bị (device drivers).
- Ưu điểm và khuyết điểm đối với linux kernel trong embedded system:
 - + Ưu điểm:
 - Mã nguồn mở và miễn phí: Không tốn chi phí bản quyền thương mại và cho phép can thiệp sâu vào nhân (kernel) để tối ưu hóa, thêm bớt tính năng.
 - Hệ sinh thái phần cứng phong phú: Hỗ trợ hầu hết các kiến trúc vi xử lý hiện nay (ARM, x86, RISC-V, MIPS) và tích hợp sẵn trình điều khiển (driver) cho rất nhiều thiết bị ngoại vi.
 - Đa nhiệm mạnh mẽ: Quản lý tiến trình (process) và bộ nhớ ảo vượt trội, rất phù hợp cho các thiết bị cần chạy nhiều tác vụ cùng lúc như AI biên, Smart TV, hay IoT gateway.
 - Mạng và giao tiếp hoàn hảo: Cung cấp sẵn các giao thức mạng (TCP/IP, Bluetooth, Wi-Fi, USB), giúp thiết bị kết nối internet và các thiết bị ngoại vi dễ dàng.
 - Bảo mật & Cộng đồng: Được cập nhật bảo mật liên tục từ cộng đồng không lồ, đi kèm các công cụ quản lý quyền truy cập mạnh mẽ.
 - + Khuyết điểm:
 - Khởi động chậm (Slow Boot time): Quá trình khởi động của Linux kernel qua nhiều bước khởi tạo khiến thời gian từ lúc cấp nguồn đến khi sẵn sàng sử dụng lâu hơn đáng kể so với hệ điều hành thời gian thực (RTOS) hoặc bare-metal.
 - Yêu cầu tài nguyên phần cứng cao: Cần vi xử lý có bộ nhớ RAM lớn (thường từ 128 MB trở lên) và bắt buộc phải có phần cứng Quản lý Bộ nhớ (MMU - Memory Management Unit).
 - Không phải là hệ điều hành thời gian thực (Non-deterministic): Nhân Linux tiêu chuẩn không đảm bảo độ trễ (latency) đủ thấp cho các hệ thống đòi hỏi tính chính xác tuyệt

đổi theo thời gian (ví dụ: bộ điều khiển động cơ, thiết bị y tế chuyên dụng). Để khắc phục, cần phải vá thêm các module như PREEMPT_RT.

- Tiêu hao năng lượng: Không phù hợp cho các thiết bị chạy bằng pin nhỏ (như thiết bị cảm biến IoT 8-bit, 16-bit) do mức tiêu thụ điện năng cao khi hệ thống phải duy trì các tiến trình ngấm.

- Một số ứng dụng thực tế mà Linux Kernel trong Embedded System đã mang lại:

+ Thiết bị mạng và viễn thông:

- Bộ định tuyến & Điểm truy cập (Routers & Access Points): Các thiết bị từ bộ phát Wi-Fi gia đình đến router doanh nghiệp lớn đều chạy Linux để xử lý định tuyến gói tin, tường lửa và VPN.
- Hệ thống trạm thu phát sóng (5G/LTE): Các thiết bị trạm gốc của mạng viễn thông sử dụng Linux thời gian thực (Real-time Linux) để xử lý tín hiệu tốc độ cao.

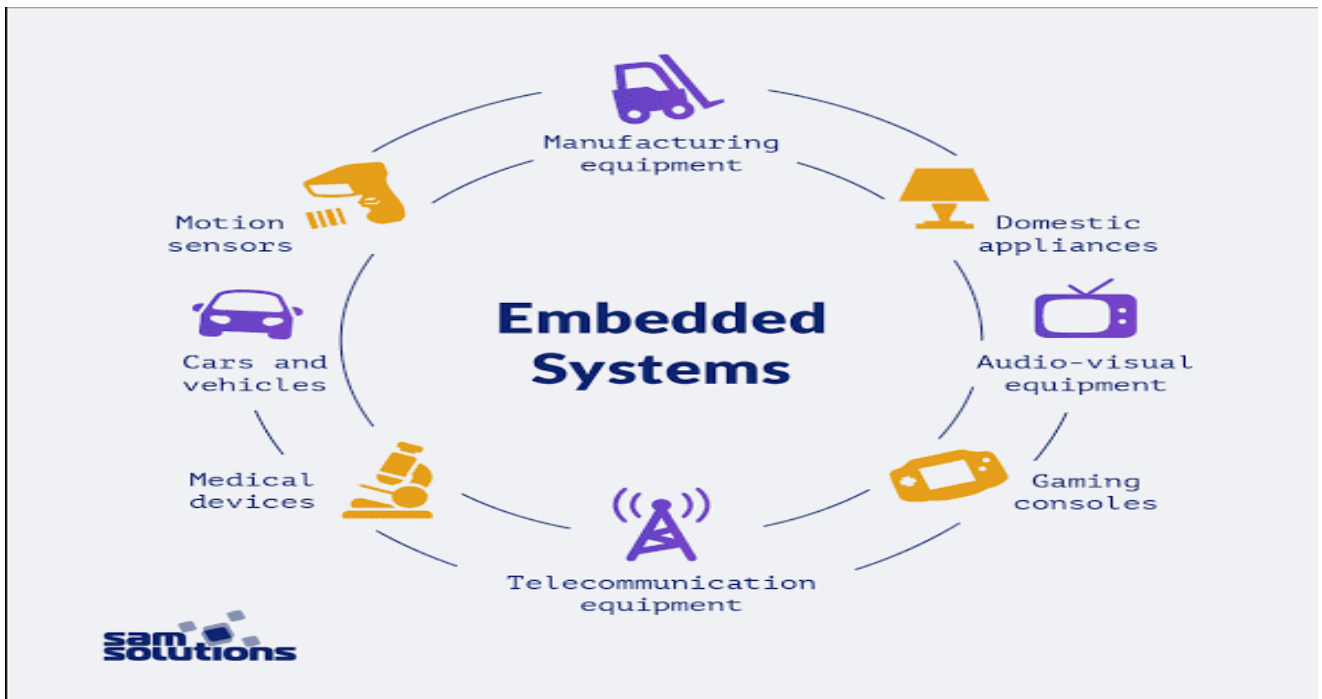
+ Thiết bị điện tử tiêu dùng:

- Smart TV & TV Box: Các hệ điều hành như Android TV (dựa trên Linux) xử lý mượt mà việc giải mã video 4K/8K và điều khiển ứng dụng trực tuyến.
- Máy ảnh kỹ thuật số: Được dùng để điều khiển cảm biến ảnh, xử lý ảnh thô (RAW) và các tính năng thông minh.

+ Thiết bị y tế:

- Thiết bị theo dõi bệnh nhân: Máy đo điện tâm đồ (ECG), máy siêu âm cầm tay hay hệ thống theo dõi giường bệnh.
- Máy hỗ trợ thở: Cần hệ điều hành có độ trễ cực thấp để điều khiển luồng khí chính xác, cứu mạng bệnh nhân.

+ Thiết bị hàng không: Nhờ khả năng chống chịu lỗi tốt và khả năng hoạt động không cần khởi động lại trong thời gian dài, Linux được Cơ quan Hàng không Vũ trụ Mỹ (NASA) và các công ty tư nhân tin dùng trong hệ thống điều khiển và đo từ xa (Vệ tinh siêu nhỏ (CubeSats) & Tàu vũ trụ).



Hình 4.4. Hệ thống nhúng (Embedded System)



Hình 4.5. Máy đo điện tâm đồ (ECG)

4.4. LINUX KERNEL TRONG IoT

- IoT hay Internet vạn vật là mạng lưới kết nối các thiết bị vật lý, đồ gia dụng, phương tiện hoặc máy móc với Internet.
- Vai trò cốt lõi của Linux Kernel trong các thiết bị IoT:

- + Giao tiếp phần cứng: Hạt nhân (kernel) sử dụng các device driver để kết nối trực tiếp bộ vi xử lý với cảm biến, module Wi-Fi, Bluetooth hoặc camera.
- + Quản lý bộ nhớ & tiến trình: Phân bổ RAM và CPU hợp lý để thiết bị hoạt động đa nhiệm mà không bị treo, ngay cả trên các phần cứng có dung lượng nhỏ.
- + Bảo mật: Cung cấp các tính năng tích hợp như quản lý quyền truy cập và cô lập tiến trình để bảo vệ thiết bị IoT khỏi các cuộc tấn công mạng.

- Ưu điểm và khuyết điểm đối với linux kernel trong IoT:

+ Ưu điểm:

- Tùy biến không giới hạn: Cho phép nhà phát triển biên dịch (compile) nhân, thêm/bớt driver và chỉ giữ lại những thành phần cần thiết để tối ưu dung lượng cho các thiết bị có tài nguyên hạn chế.
- Hỗ trợ phần cứng khổng lồ: Tương thích sẵn với hàng ngàn vi điều khiển và module ngoại vi, giúp tiết kiệm thời gian viết trình điều khiển (driver) từ đầu.
- Hiệu năng và ổn định: Xử lý đa nhiệm mượt mà, quản lý tiến trình và mạng (networking stack) hiệu quả, đáp ứng tức thời ngay cả trong thời gian thực.
- Hệ sinh thái phong phú: Sở hữu cộng đồng lập trình viên lớn hỗ trợ và hàng loạt framework mã nguồn mở dành riêng cho phát triển IoT.

+ Khuyết điểm:

- Yêu cầu phần cứng cao (Resource Heavy): Kernel Linux và hệ thống tệp cơ bản chiếm khá nhiều dung lượng bộ nhớ. Do đó, nó không phù hợp với các thiết bị vi điều khiển (MCU) giá rẻ hoặc bị giới hạn nặng về RAM/ROM (dưới vài megabyte).
- Thời gian khởi động chậm: So với việc bật và chạy ngay lập tức của vi điều khiển, Linux mất nhiều thời gian hơn để khởi động (boot time) do phải tải Kernel và khởi tạo hàng loạt tiến trình.
- Khó khăn trong cập nhật (OTA - Over-The-Air): Đối với các phiên bản Linux nhúng quá cũ hoặc quá cồng kềnh, việc cập nhật bảo mật qua mạng cho thiết bị vật lý ở xa là một thách thức lớn (thiếu dung lượng bộ nhớ đệm, rủi ro lỗi hệ thống).

- Những ứng dụng thực tế đối với Linux Kernel trong IoT:

+ Cổng IoT:

- Định tuyến và chuyển đổi giao thức: Linux kernel cung cấp ngăn xếp mạng (network stack) mạnh mẽ, tích hợp sẵn các giao thức như TCP/IP, MQTT, CoAP, BLE, Zigbee để định tuyến và chuyển đổi dữ liệu dễ dàng.
 - Bảo mật mạng (Firewall): Ứng dụng các công cụ như iptables hoặc nftables có sẵn trong nhân để cấu hình tường lửa, bảo vệ hệ thống khỏi các truy cập trái phép.
- + Tự động hóa công nghiệp:
- Xử lý thời gian thực (Real-Time Linux): Sử dụng các bản vá như PREEMPT_RT để nhân Linux phản hồi tức thì các ngắt phần cứng, đảm bảo độ trễ gần như bằng 0 khi điều khiển cánh tay robot, băng chuyền hoặc máy CNC.
 - PLC dựa trên phần mềm (Soft-PLC): Biến các máy tính nhúng chạy Linux thành các bộ điều khiển logic lập trình để tự động hóa nhà máy.
- + Thành phố thông minh & Tòa nhà thông minh:
- Giám sát môi trường & Giao thông: Các thiết bị dùng bo mạch như Raspberry Pi chạy các bản phân phối Linux thu thập dữ liệu từ camera AI, cảm biến chất lượng không khí, cảm biến đỗ xe và truyền thông tin theo thời gian thực về trung tâm điều hành.
 - Hệ thống tự động hóa tòa nhà: Dùng các nền tảng tự động hóa mã nguồn mở chạy trên nhân Linux (như Home Assistant hay Node-RED) để quản lý chiếu sáng, điều hòa và an ninh.



Hình 4.6. Internet vạn vật (Internet of Things - IoT)

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. KẾT LUẬN

Qua quá trình tìm hiểu và thực hiện đồ án với đề tài là tìm hiểu về Linux Kernel, đề tài đồ án đã giúp làm rõ vai trò và tầm quan trọng của kernel trong hệ điều hành Linux. Linux Kernel là thành phần cốt lõi, chịu trách nhiệm quản lý tài nguyên phần cứng và cung cấp môi trường hoạt động cho các chương trình ứng dụng.

Đề tài đồ án đã trình bày được các nội dung quan trọng và cơ bản. Cụ thể:

- Khái niệm và lịch sử phát triển của Linux Kernel.
- Kiến trúc tổng quát của Linux Kernel.
- Các chức năng chính và cơ bản của Linux Kernel như quản lý tiến trình, quản lý bộ nhớ
- Tìm hiểu được khái niệm, một số chức năng chính, nguyên tắc và cách thức bảo mật trong Linux Kernel.
- Tìm hiểu ứng dụng của Linux Kernel trong thực tế, nhất là server (máy chủ), embedded system (hệ thống nhúng) và IoT (Internet vạn vật).

Từ đó có thể thấy Linux Kernel là một nhân hệ điều hành ổn định, mạnh mẽ, linh hoạt và độ mở cao, được sử dụng nhiều và rộng rãi trong nhiều lĩnh vực, nhất là công nghệ thông tin hiện nay.

5.2. HẠN CHẾ

Mặc dù đã được những kết quả nhất định nhưng do thời gian làm đồ án có hạn nên đề tài vẫn có một số hạn chế như:

- Nội dung nghiên cứu chủ yếu dừng lại ở mức tổng quan, chưa đi sâu vào mã nguồn và cơ chế hoạt động chi tiết của Linux Kernel.
- Chưa thực hiện việc biên dịch (compile) và tùy chỉnh Linux Kernel trên môi trường thực tế.
- Chưa đánh giá hiệu năng và khả năng tối ưu hóa của Linux Kernel trên các hệ thống khác nhau.
- Phạm vi nghiên cứu còn giới hạn, chưa đề cập sâu đến các công nghệ mới như Container, Docker, Kubernetes hay các cơ chế bảo mật nâng cao.
- Chưa nghiên cứu đầy đủ các thành phần nâng cao như Kernel Module, Device Driver,

Scheduler, Virtual Memory và Network Stack.

5.3. HƯỚNG PHÁT TRIỂN

Trong thời gian tới, đề tài cần tiếp tục mở rộng và phát triển để làm đồ án tốt hơn và hoàn thiện hơn:

- Nghiên cứu chuyên sâu về kiến trúc Linux Kernel:
 - + Quản lý tiến trình (Process Management).
 - + Quản lý bộ nhớ (Memory Management).
 - + Hệ thống tệp (File System).
 - + Network Stack và Device Driver.
- Nghiên cứu về cơ chế bảo mật Linux Kernel:
 - + SELinux và AppArmor.
 - + Namespace và Cgroups.
 - + Access Control List (ACL).
- Nghiên cứu Linux Kernel trong môi trường ảo hóa và điện toán đám mây:
 - + Docker và Container.
 - + Kubernetes.
- Thực hiện các thử nghiệm thực tế:
 - + Biên dịch và tùy chỉnh Linux Kernel.
 - + Tối ưu hóa CPU, bộ nhớ và hệ thống I/O.
- Ứng dụng Linux Kernel trong hệ thống nhúng và IoT:
 - + Tùy biến nhân Linux cho các thiết bị nhúng.
 - + Phát triển trình điều khiển thiết bị (Device Driver).
- Tìm hiểu và phát triển Kernel Module:
 - + Xây dựng các mô-đun nhân Linux đơn giản.
 - + Tìm hiểu cơ chế nạp và gỡ bỏ module.
 - + Tìm hiểu những ưu điểm và khuyết điểm của Kernel Module.
 - + Tìm hiểu các bảo vệ Kernel Module.

TÀI LIỆU THAM KHẢO

- [1] Steven J. Vaughan-Nichols, "It's a Linux-powered car world", 2019.
- [2] Swapnil Bhartiya, "Best Linux distros of 2016: Something for everyone", 2016.
- [3] Linus Torvalds, "Hybrid kernel, not NT", 2006.
- [4] Steven J. Vaughan-Nichols, "It's a Linux-powered car world", 2019.
- [5] Douglas M. Wells, "A Trusted, Scalable, Real-Time Operating System Environment", 1994.
- [6] Douglas M. Wells, "A Trusted, Scalable, Real-Time Operating System Environment", 1994.
- [7] Jorrit N Herder, "Toward a True Microkernel Operating System", Vrije Universiteit Amsterdam, 2005.